Cleaner: An Autonomous Scheduling Framework for Time-Series Data Cleaning

Abstract

Time series often suffer from heterogeneous and evolving errors that degrade downstream task performance. Although numerous time series cleaning (TSC) methods have been proposed, they are typically tailored for specific error assumptions. Recent studies improve cleaning robustness by integrating multiple TSC methods, yet they rely on fixed integration strategies and cannot adapt to unseen error types or newly available TSC methods. Flexibly composing diverse TSC methods remains labor-intensive and may introduce inconsistent repairs, over-cleaning, or redundant computational overhead. This paper proposes Cleaner, an autonomous cleaning framework that adaptively composes and schedules TSC methods to handle heterogeneous and evolving errors. (1) Cleaner organizes existing TSC methods into a Cleaning Path spanning two paradigms: coupled cleaning, which performs unified cleaning without explicitly localizing errors, and decoupled cleaning, which separates cleaning into detection and repair stages, enabling flexible composition of diverse TSC methods. (2) It leverages reinforcement learning to dynamically select and schedule appropriate methods along the Cleaning Path. (3) It designs a Discriminator to regulate the scheduling process, preventing both premature termination and over-cleaning. (4) It develops an Adaptation Mechanism to generalize to new datasets, error types, and TSC methods. Extensive experiments on six datasets against 24 baselines and variants demonstrate the superiority of Cleaner in cleaning effectiveness, efficiency, and downstream task performance.

CCS Concepts

• **Information systems → Data cleaning**.

Keywords

Data cleaning, Time series, Autonomous scheduling

ACM Reference Format:

. 2018. Cleaner: An Autonomous Scheduling Framework for Time-Series Data Cleaning. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Background. Time series (TS) data are critical for decision-making in domains such as industry, healthcare, and aviation [14, 16, 51]. However, TS data are often corrupted by errors caused by environmental disturbances (e.g., electromagnetic noise and temperature fluctuations), equipment aging, or sensor miscalibration, which can compromise downstream analytics and potentially lead to unsafe or biased decisions [12, 22, 41].

Numerous time series cleaning (TSC) methods have been developed to detect and repair these errors, such as smoothing-based, constraint-based, and statistical approaches [19, 40, 50, 52].

Motivation. Despite extensive research, time series cleaning remains challenging due to the heterogeneous and evolving characteristic of errors. Time series may simultaneously contain diverse error types, including noise, drift, abrupt changes, gradual changes, and volatility, each exhibiting distinct characteristics [10, 11, 49, 54]. For example, noise typically appears as high-frequency, low-amplitude disturbances that undermine smoothness and interpretability [2, 18, 25], whereas drift reflects systematic deviations that accumulate over time and are often mistaken for genuine trends [45, 53]. Moreover, errors evolve over time, further increasing the difficulty of time series cleaning [46].

Owing to these diverse error characteristics, existing TSC methods are typically specialized for particular error types and adopt different cleaning strategies. For instance, smoothing-based methods primarily exploit local temporal continuity to remove noise [23, 51], while statistical methods capture data distribution to focus on volatility errors [5, 22, 50]. Constraint-based approaches, in contrast, leverage structural patterns derived from column-wise (temporal) dependencies or row-wise (inter-variable) relationships to handle volatility [42, 48] or different error types. Specifically, column-wise constraints are effective for volatility errors [42, 48], whereas row-wise constraints suit drift errors [16]. Consequently, no single TSC method can effectively handle heterogeneous and evolving errors.

Several recent studies [7, 11, 12, 14, 36] improve cleaning robustness by integrating multiple TSC methods [10]. For example, [7] combines row-wise and column-wise constraints to handle drift, gradual, and abrupt errors. However, these approaches rely on predefined integration strategies, limiting their ability to adapt to heterogeneous and evolving errors or flexibly accommodate newly available TSC methods. *Example 1* illustrates these limitations.

Example 1. Table 1(a) shows a snippet from the real-world IDF dataset [29], where red values denote erroneous data and green values indicate the ground truth. Multiple heterogeneous error types coexist and evolve over time in IDF, e.g., a gradual error in CT112 from t_5 to t_{11}, a volatility error in CT143 at t_1-t_2, and an abrupt error in CT162 from t_6 to t_{11}.

Table 1(b) reports the results of three representative TSC methods, namely, MTSClean [10] (M_1), Clean4MTS [7] (M_2) and Speed+Acc(G) [39] (M_3), under different error types. For a fair comparison, we inject gradual, volatility, or abrupt errors into randomly selected columns at sampled positions following [10]. The results show that no single TSC method consistently performs best across different error types, motivating the need for method composition.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

To systematically organize existing TSC methods, we categorize them into two paradigms: *Coupled cleaning*, which performs unified cleaning without explicitly identifying error locations; and *decoupled cleaning*, which explicitly separates error detection and repair. In this *Example*, M_3 follows the *coupled cleaning*, whereas M_1 and M_2 follow the decoupled paradigm and can be represented as $(M_1 = (M_{1d}, M_{1r}), M_2 = (M_{2d}, M_{2r}))$, where M_d and M_r denote the detection and repair modules, respectively. Consequently, numerous cleaning pipelines can be formed by flexibly composing coupled methods and decoupled modules, resulting in a large search space for method selection and scheduling.

Table 1: (a) IDF Dataset example and (b) method effectiveness

Timestamp	CT112	CT113	CT143	CT162
t_1	52.449	53.029	46.722(47.349)	47.692
t_2	52.450	53.036	46.641(47.346)	47.681
t_3	52.450	53.044	47.344	47.884
t_4	52.451	53.052	47.341	47.887
t_5	52.178(52.455)	53.046	47.360	47.906
t_6	52.344(52.440)	53.046	47.363	48.216(47.908)
t_7	52.274(52.453)	53.029	47.339	47.888(47.911)
t_8	52.434(52.445)	53.050	47.342	48.032(47.913)
t_9	52.854(52.435)	53.050	47.344	47.980(47.916)
t_{10}	52.724(52.430)	53.039	47.346	47.947(47.918)
t_{11}	52.633(52.435)	53.027	47.348	47.908(47.920)

Error types	Methods	$F1\uparrow$	$MAE\downarrow$	$RRA\uparrow$
Abrupt	M_1	**0.857**	**0.007**	**0.812**
	M_2	0.829	0.010	0.779
	M_3	0.709	0.012	0.639
Gradual	M_1	0.738	0.009	0.485
	M_2	**0.800**	**0.006**	**0.700**
	M_3	0.666	0.009	0.489
Volatility	M_1	0.763	0.007	0.570
	M_2	0.724	0.008	0.454
	M_3	**0.823**	**0.004**	**0.774**

Challenges. However, flexibly composing TSC methods introduces the following challenges. First, the large space of cleaning pipelines makes exhaustive exploration computationally infeasible. For instance, even with only three candidate methods in *Example 1*, there are 15 possible combinations following $\sum_{k=1}^{n} P(n,k)$, where n and k denote the numbers of available and selected methods, respectively. The decomposition of decoupled methods into detection and repair components further enlarges the search space, making effective method selection increasingly difficult. Second, as error types evolve over time, fixed pre-selected cleaning methods may become unsuitable. Continuing to execute them under a static scheduling strategy may lead to over-cleaning or incidental errors, contaminating the original clean data and degrading data quality [14, 35]. Finally, an optimal cleaning solution for one time series often fails to generalize to others or even to different periods of the same time series, since different TS datasets exhibit diverse patterns [21]. Moreover, as newly available TSC methods continue to emerge, enabling their plug-and-play integration into existing cleaning framework remains challenging.

To address these challenges, we initiate the study of the **Autonomously Scheduled Cleaning** (ASC) problem. Instead of applying predefined TSC methods with fixed strategies, ASC constructs an adaptive cleaning pipeline by abstracting coupled and decoupled methods into a unified operator space and optimizing operator selection, scheduling, and termination. By transforming cleaning from a fixed-method paradigm into an adaptive pipeline optimization problem, ASC enables effective handling of heterogeneous and evolving errors while supporting the plug-and-play integration of diverse TSC methods.

Methodology. To effectively solve the ASC problem, we propose Cleaner, an autonomous framework that schedules TSC methods and constructs cleaning pipelines for heterogeneous and evolving errors. Cleaner consists of three key components. First, it organizes existing TSC methods into a unified Cleaning Path, where each method is instantiated as a cleaning operator, and leverages reinforcement learning (RL) to dynamically schedule appropriate operators across coupled and decoupled cleaning paradigms. Second, it introduces a Discriminator to regulate the scheduling process, preventing premature termination and over-cleaning. Third, it develops an Adaptation Mechanism to generalize across new datasets, error types, and newly available TSC methods.

Table 2: Notations

Symbol	Description
M, M_i	A set of candidate time series cleaning methods and its i-th method
D, D', D^*	An observed time series dataset, its cleaned version, and the unavailable clean version
T, t	The total timestamps of the time series and its t-th timestamp
x_t	An observation at the t-th timestamp in D
x_t^m	An observed value of the m-th variable at timestamp t
$\mathcal{W}, W_k, W'_k$	A sequence of disjoint windows, the k-th window of length l on M variables, and its cleaned window
$\mathcal{P}$	A cleaning pipeline for D
σ, Γ	The scheduling policy and the termination criterion
$O, O_{k,i}$	The candidate operator space and the i-th operator on the k-th window

We implement Cleaner with 19 representative coupled and decoupled TSC methods and evaluate it on six datasets against 24 baselines and variants. Results verify its superiority in cleaning effectiveness, efficiency, and downstream performance, being comparable to an oracle with optimal per-window selection.

Our **contributions** are summarized as follows.

(1) We formulate the **Autonomously Scheduled Cleaning** (ASC) problem, shifting time series cleaning from individual methods to autonomous cleaning pipelines.

(2) We propose Cleaner, an autonomous time series cleaning framework that unifies diverse TSC methods into a Cleaning Path and employs RL to dynamically schedule coupled and decoupled methods to handle heterogeneous and evolving errors.

(3) We introduce a Discriminator to regulate the scheduling process and prevent premature termination and over-cleaning.

(4) We develop an Adaptation Mechanism to generalize across datasets, evolving error types, and newly available TSC methods.

2 Preliminaries

In this section, we first describe the main concepts and then formulate the problem definition. Notations are summarized in Table 2.

Definition 1. ***Time Series (TS).*** *A time series dataset $D = (x_1, x_2, \ldots, x_T)$ consists of observations over T timestamps. At each timestamp $t \in [1, T]$, the observation $x_t = \{x_t^1, x_t^2, \ldots, x_t^M\}$ represents the values of M variables, where $x_t \in \mathbb{R}^M$ and x_t^m denotes the observation of the m-th ($m \in [1, M]$) variable at timestamp t. If $M = 1$, D is a univariate time series; otherwise, it is multivariate.*

Definition 2. ***Window.*** *A window is a projection of D over a consecutive time interval $[t, t+l-1]$, denoted by $W[t, t+l-1]$, where $1 \le t < t+l-1 \le T$. Given a time series D, we partition it into a sequence of disjoint windows $\mathcal{W} = (W_1, \ldots, W_N)$. For brevity, we use W, when there is no ambiguity.*

Throughout this paper, a window serves as the basic unit for cleaning decisions, as processing the entire time series is impractical [42]. Window-level processing enables Cleaner to adapt cleaning methods to heterogeneous and evolving errors across windows.

Heterogeneous Errors. Errors in time series data can exhibit diverse patterns. We consider five representative error types commonly observed in real-world applications: (1) Abrupt change (AC) [42, 48]: sudden jumps at a single or a few timestamps that sharply deviate from the normal range; (2) Gradual change (GC) [7]: short-term transition where values gradually shift and stabilize at a new level;

Table 3: Taxonomy of cleaning methods with properties

Paradigm	Method	Category	Functional Decoupling		Composable	Suitable Scenario
			Explicit Localization	Targeted Repair		
Coupled cleaning (Unified detection and repair)	SWAB [20, 25]	Smoothing	✗	✓	✗	NS
	SpCh [45]	Statistical	✗	✓	✗	VA
	MA [2]	Smoothing	✗	✓	✗	NS
	LsGreedy [50]	Statistical	✗	✓	✗	VA
	WMA [55]	Smoothing	✗	✓	✗	NS
	Speed(G) [42]	Constraint	✗	✓	✗	VA
	EWMA [17, 18]	Smoothing	✗	✓	✗	NS
	Speed+Acc(G) [39]	Constraint	✗	✓	✗	VA
Decoupled cleaning (Separated detection and repair)	Speed(L) [42]	Constraint	✓	✓	✓	VA
	Speed+Acc(L) [39]	Constraint	✓	✓	✓	VA
	Vari [48]	Constraint	✓	✓	✓	VA
	MTSClean [10]	Constraint	✓	✓	✓	VA, DF, AC
	Clean4MTS [7]	Constraint	✓	✓	✓	DF, AC, GC
	Linear interp. [30]	Statistical	✗	✓	✓	DF
	KNN [37]	Statistical	✗	✓	✓	NS
	STL [4]	Statistical	✗	✓	✓	VA
	LOF [1]	Statistical	✓	✗	✓	AC
	DBSCAN [34]	Statistical	✓	✗	✓	VA
	Z-Score [3]	Statistical	✓	✗	✓	AC

(3) Drift (DF) [16]: long-term deviation accumulating in one direction, often mistaken for genuine trends; (4) Volatility (VA) [5, 22, 50]: unstable yet structured oscillations of varying magnitude around the normal value; (5) Noise (NS) [23, 51]: unstructured small-scale perturbations lacking temporal correlation. These error types may co-occur and evolve over time, resulting in heterogeneous error patterns. Although not exhaustive, they represent common error types studied in time series cleaning.

Cleaning Methods. Time series cleaning consists of two stages: error detection and repair. Based on how these stages are integrated, we categorize existing cleaning methods into two paradigms.

– **Coupled cleaning** $\mathcal{M}_c$, which implicitly entangles detection and repair into a unified process. For example, autoregressive smoothing [20, 25] directly repairs the corrupted values without explicitly identifying error locations.

– **Decoupled cleaning** $(\mathcal{M}_d, \mathcal{M}_r)$, which explicitly separates detection and repair into two phases. A detection method $M_d \in \mathcal{M}_d$ first identifies errors, and a repair method $M_r \in \mathcal{M}_r$ repairs them. These two phases can be flexibly combined. For instance, LOF [1] can be followed by linear interpolation [30] for repair.

Then, we define the set of candidate cleaning methods as $\mathcal{M} = \{M_1, \ldots, M_n\}$. Each candidate is either a coupled method $M_c \in \mathcal{M}_c$ or a decoupled method represented as a pair (M_d, M_r), where $M_d \in \mathcal{M}_d$ and $M_r \in \mathcal{M}_r$. We integrate 19 representative cleaning methods, as summarized in Table 3, and categorize them into coupled and decoupled paradigms accordingly.

Traditional Time Series Cleaning (TSC) typically assumes a predefined cleaning method M_i for a contaminated time series. During cleaning, the series is partitioned into a sequence of windows $W_1, W_2, \ldots, W_K$, and the fixed method M_i is uniformly applied to each window W_k ($k \in [1, K]$) to produce the cleaned series D'. However, applying a single predefined method to all windows limits the ability to handle heterogeneous errors and adapt to evolving error patterns over time.

Problem Definition. Autonomously Scheduled Cleaning (ASC). Given a contaminated time series D and a set of candidate cleaning methods $\mathcal{M}$, ASC aims to construct a cleaning pipeline that adaptively cleans D window by window. Following Definition 2, D is partitioned into disjoint windows $(W_1, \ldots, W_N)$. Each candidate method acts as a cleaning operator on a window: a coupled method $M_c \in \mathcal{M}_c$ directly performs cleaning, while a decoupled detection-repair pair $(M_d, M_r) \in \mathcal{M}_d \times \mathcal{M}_r$ forms a composite operator $M_r \circ M_d$, where $\circ$ denotes operator composition. Therefore, the candidate operator space is defined as:

$$\mathcal{O} = \mathcal{M}_c \cup \{M_r \circ M_d \mid M_d \in \mathcal{M}_d,\ M_r \in \mathcal{M}_r\}.$$

An ASC pipeline is defined as $\mathscr{P} = (\mathcal{O}_s, \sigma, \Gamma)$, where $\mathcal{O}_s \subseteq \mathcal{O}$ is the set of selected operators, σ is a scheduling policy among $\mathcal{O}_s$, and Γ determines whether to stop cleaning the current window and proceed to the next. Executing $\mathscr{P}$ on a window W_k produces a cleaning sequence $(O_{k,1}, O_{k,2}, \ldots, O_{k,n_k})$, where $n_k \leq C$ is bounded by the iteration budget. The resulting cleaned window is:

$$W'_k = O_{k,n_k} \circ \cdots \circ O_{k,1}(W_k),$$

and the cleaned time series is obtained as:

$$D' = \mathscr{P}(D) = \text{Merge}(W'_1, \ldots, W'_N).$$

Let D^* denote the ground-truth clean time series underlying D, and let $\Delta(\cdot, \cdot)$ quantify the discrepancy between D^* and D (e.g., MSE or MNAD in §8). Although D^* is generally unavailable in practice, it defines the ideal cleaning objective. Accordingly, ASC aims to find an optimal cleaning pipeline:

$$\mathscr{P}^* = \arg\min_{\mathscr{P}} \Delta(\mathscr{P}(D), D^*) \quad \text{s.t. } n_k \leq C, \forall k \in [1, N].$$

3 Cleaner overview

This section presents Cleaner, an autonomous cleaning framework for the ASC problem that transforms diverse TSC methods into executable cleaning operators and dynamically constructs cleaning pipelines for heterogeneous and evolving errors. As illustrated in Figure 1, Cleaner consists of three core modules: the Cleaning Path, the Discriminator, and the Adaptation Mechanism.

Cleaning Path. To enable flexible composition of TSC methods from different cleaning paradigms (Table 3), we design the Cleaning Path to organize both coupled and decoupled cleaning methods. Specifically, it employs two specialized agents: a coupled agent for direct cleaning; and a decoupled agent for two-phase detection and repair. It then leverages reinforcement learning to learn agent-specific cleaning policies, while a reward-driven scheduler coordinates the coupled and decoupled agents to select appropriate cleaning operators for each window under heterogeneous and evolving errors. Through iterative scheduling, the Cleaning Path progressively constructs a cleaning pipeline.

Discriminator. The Discriminator then regulates the scheduling process to prevent premature termination or over-cleaning. At each scheduling step, it determines whether the current window should continue cleaning or proceed to the next window. Specifically, the Discriminator jointly considers two complementary decision signals: *quality evidence*, which reflects cleaning quality, and *progress evidence*, which indicates remaining cleaning budget, to make adaptive scheduling decisions throughout the cleaning process.

Adaptation Mechanism. To improve generalization across new datasets, error types, and TSC methods, Cleaner develops a meta-learning-based Adaptation Mechanism with two components: (1) *Knowledge Integration*, which learns transferable cleaning knowledge across multiple datasets; (2) *Model Adaptation*, which efficiently specializes the learned knowledge to unseen datasets through few-shot fine-tuning. Moreover, it supports plug-and-play

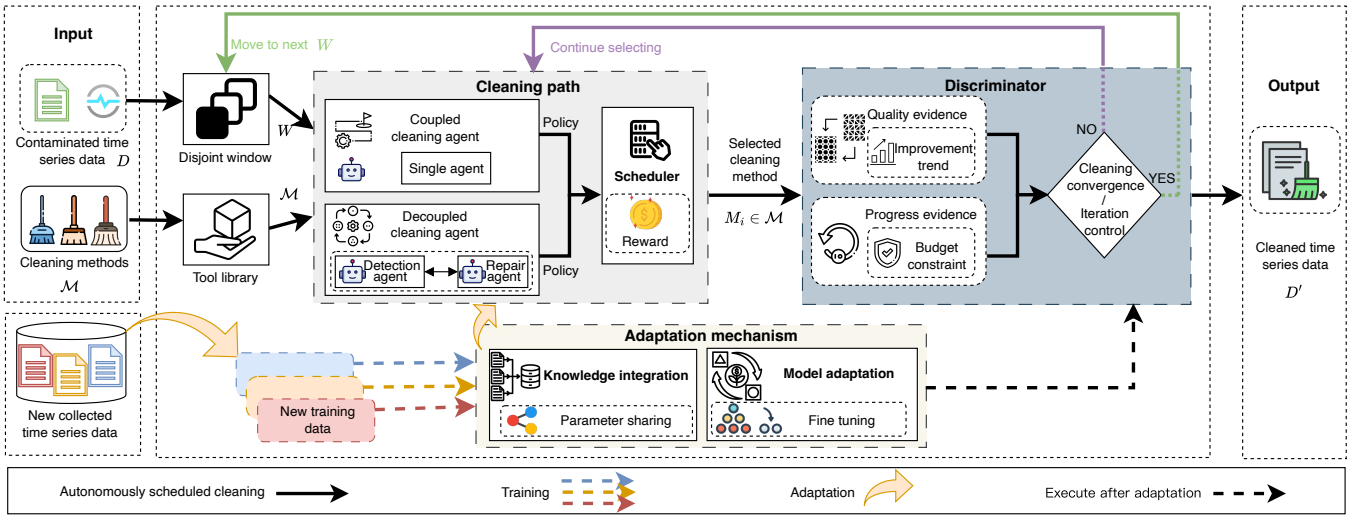


Fig. 1: An overview of Cleaner

integration of newly available TSC methods by efficiently adapting the cleaning policy without retraining the entire framework.

Workflow. As illustrated in Figure 1, Cleaner takes a contaminated time series D and a set of candidate cleaning methods $\mathcal{M}$ as input and operates window-wise. For each window W, the Cleaning Path observes its data representation and schedules suitable methods from $\mathcal{M}$ to handle heterogeneous and evolving errors. The Discriminator decides whether to continue processing the current window or proceed to the next. This process repeats until all windows are processed, producing the cleaned time series D'.

4 Cleaning Path

This section details the Cleaning Path in Cleaner. We first present its design rationale and then describe its main components.

4.1 Rationale

The Cleaning Path is designed to autonomously construct a cleaning pipeline by determining which cleaning methods to apply and in what order. Since each cleaning decision modifies the data state and consequently influences subsequent decisions, the cleaning process can be naturally formulated as a Markov Decision Process (MDP). Accordingly, the Cleaning Path employs reinforcement learning (RL) to learn an effective cleaning policy.

Although several RL-based approaches have been explored for data cleaning [35, 36, 38], they remain insufficient for ASC. Our Cleaning Path differs from prior approaches in three aspects. First, unlike existing methods that represent the state using error sets or static data-quality features, the Cleaning Path learns temporal-variable representations from time-series windows with a dedicated TVACN (Temporal-Variable Actor-Critic Network), enabling state-aware scheduling. Second, existing methods typically treat candidate actions as homogeneous choices, making them difficult to coordinate heterogeneous cleaning paradigms. In contrast, the Cleaning Path explicitly models coupled and decoupled cleaning behaviors with dedicated agents and dynamically coordinates them through unified scheduling. Third, existing methods typically rely on immediate or relatively stationary rewards. In contrast, the Cleaning Path adopts an Actor–Critic architecture, where the Critic estimates long-term cleaning utility to guide the cleaning decisions under evolving errors.

As illustrated in Figure 2, the Cleaning Path consists of four key components: TVACN, which learns temporal-variable representations and the cleaning policy; the coupled agent, which select a coupled cleaning method M_c; the decoupled agent, which generates a decoupled cleaning method (M_d, M_r); and the scheduler, which coordinates these agents to progressively construct a cleaning pipeline. Below, we first introduce the RL formulation of the Cleaning Path, and then present its four components in detail.

4.2 RL formulation

The Cleaning Path leverages reinforcement learning (RL) to guide method selection. We briefly describe how each element of the RL is instantiated in the Cleaning Path.

(1) *Environment.* The environment corresponds to the cleaning process over the contaminated time series D, whose data quality (i.e., state) evolves with sequentially applied cleaning actions.

(2) *State.* The state $s \in S$ represents the current window being cleaned. It is encoded by TVACN into a temporal-variable representation that captures both temporal and variable relationships.

(3) *Action.* The action $a \in A$ corresponds to a selected cleaning operator O_i from the candidate operator space $\boldsymbol{O}$. Executing action a applies O_i to the current state s. The action space is discrete yet heterogeneous, covering both coupled operators and decoupled detection-repair operators.

(4) *Transition.* The transition function $\mathcal{T}(s'|s,a)$ describes how the state changes after the action a. The transition is inherently non-stationary due to evolving error distributions.

(5) *Reward.* The reward $r = R(s,a,s')$ quantifies the cleaning gain achieved after executing action a, guiding the policy towards effective scheduling decisions.

(6) *Agent.* The agent is an action generator implemented using a Temporal-Variable Actor–Critic Network (TVACN) (details in §4.3),

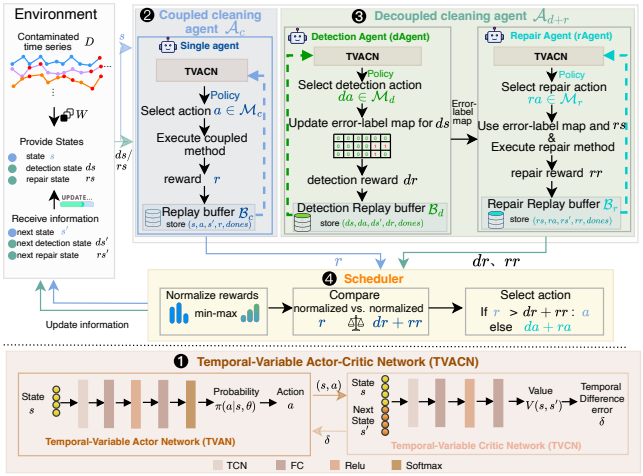


Fig. 2: Cleaning path

which receives the current state from the environment and the resulting reward, and updates its policy to select the next action.

In the following, "method", "operator", and "action" are used interchangeably when referring to cleaning process in CLEANER.

4.3 TVACN

TVACN consists of a Temporal-Variable Actor Network (TVAN) and a Temporal-Variable Critic Network (TVCN), as illustrated in Figure 2 ❶. The TVAN serves as the policy network, while the TVCN acts as the value network to guide policy learning. Together, they learn the adaptive cleaning policy for method selection.

Temporal-Variable Actor Network (TVAN). The TVAN is responsible for learning a policy $\pi(a|s,\theta)$ to select cleaning actions by mapping the current state s to a probability distribution over candidate actions. Specifically, it adopts a TCN+MLP+Softmax architecture: $\pi(a|s,\theta) = \text{softmax}(\text{FC}(\text{ReLU}(\text{FC}(\text{TCN}(s)))))$. The TCN (Temporal Convolutional Network) first captures temporal and variable representations from s, while the extracted representations are then transformed by two fully connected (FC) layers into high-level decision features, followed by a Softmax function to output the selection probability of each candidate cleaning method.

During policy learning, the TVAN samples cleaning actions according to $\pi(a|s,\theta)$, and it is optimized through the policy gradient objective: $\mathcal{L}_{TVAN} = -\mathbb{E}[\log \pi(a|s,\theta) \cdot \delta]$, where $\mathbb{E}[\cdot]$ denotes the expectation over sampled state-action pairs, δ is the temporal-difference (TD) error provided by TVCN. A positive TD error indicates that the selected action achieves better performance than expected, increasing the probability of selecting the actions in similar states. Conversely, a negative TD error reduces their probabilities.

Temporal-Variable Critic Network (TVCN). TVCN estimates the expected cumulative reward of the current state to evaluate the long-term effectiveness of selected actions. It adopts a TCN+MLP architecture: $V(s) = \text{FC}(\text{ReLU}(\text{FC}(\text{TCN}(s))))$, which extracts state representations and maps them to a scalar value estimate.

After executing the selected cleaning action a and obtaining the next state s', the target value is calculated as: $V_{target} = r + \gamma V(s')(1 - dones)$, where r denotes the immediate reward, γ is the discount factor, and $dones$ indicates whether the current cleaning process terminates (details in §5). And the temporal-difference (TD) error δ is then computed as: $\delta = V_{target} - V(s)$, which quantifies the deviation between the actual return after executing the action and the estimated value. This TD error is used to update the TVCN and provide feedback for optimizing TVAN. Thus, the TVCN is optimized by minimizing the expectation over squared TD error: $\mathcal{L}_{TVCN} = \mathbb{E}\left[\delta^2\right]$.

Collaboration between TVAN and TVCN. The TVAN refines its policy based on the accurate TD error provided by the TVCN, while the TVCN improves its value estimation using the state–action pairs collected by TVAN. This continuous interaction enables TVACN to progressively learn and optimize the cleaning policy.

Based on the learned cleaning policy, TVACN drives two specialized agents to select candidate cleaning methods under different paradigms. We next describe the coupled cleaning agent (§4.4) and the decoupled cleaning agent (§4.5).

4.4 Coupled cleaning agent $\mathcal{A}_c$

To support the coupled cleaning methods $\mathcal{M}_c$, we design the coupled cleaning agent $\mathcal{A}_c$, which adopts a single-agent RL architecture with TVACN as the policy learning backbone to select appropriate coupled cleaning methods, as illustrated in Figure 2 ❷.

Specifically, given a window W that potentially contains errors, the agent observes the current state s that encodes the data condition and error characteristics. Based on the policy learned by TVACN, the agent selects an action a corresponding to a coupled cleaning method $M_c \in \mathcal{M}_c$. After executing the selected method, the environment updates the data state and returns the next state s', reward r, and quality-termination signal $dones$. The interaction tuple $\tau_c = (s, a, s', r, dones)$ is stored in the replay buffer $\mathcal{B}_c$ for coupled policy optimization.

Through continuous interactions, $\mathcal{A}_c$ learns an adaptive coupled policy for different coupled cleaning methods. Instead of relying on predefined execution orders, it dynamically determines which coupled methods should be applied and in what sequence according to evolving error patterns.

4.5 Decoupled cleaning agent $\mathcal{A}_{d+r}$

To support the decoupled cleaning methods $\mathcal{M}_d + \mathcal{M}_r$, we design a decoupled cleaning agent $\mathcal{A}_{d+r}$ that coordinates detection and repair through a dual-agent RL architecture. The dAgent (detection agent) identifies errors and updates the error-label map, which guides the rAgent (repair agent) in selecting appropriate repair methods, facilitating a cooperative detection-repair process, as shown in Figure 2 ❸.

Specifically, given the window W, the dAgent observes the detection state ds and uses its TVACN-based policy to select a detection action da corresponding to a detection method $M_d \in \mathcal{M}_d$. After executing da, the environment updates the error-label map and returns the next detection state ds', reward dr and quality-termination signal $dones$. The detection transition tuple $\tau_d = (ds, da, ds', dr, dones)$ is stored in the detection replay buffer $\mathcal{B}_d$ for optimizing the detection policy. Based on the updated error-label map, the rAgent observes the repair state rs and uses its TVACN to select a repair action $ra \in \mathcal{M}_r$ to correct the identified errors. After the repair action is executed, the environment returns the next repair state

rs', reward rr, and quality-termination signal $dones$. The repair transition tuple $\tau_r = (rs, ra, rs', rr, dones)$ is stored in the repair replay buffer $\mathcal{B}_r$ to optimize the repair policy.

Through the cooperative optimization between dAgent and rAgent, $\mathcal{A}_{d+r}$ learns effective policies to coordinate heterogeneous detection and repair methods, automatically determining the appropriate detection-repair combinations for evolving error patterns.

4.6 Scheduler

To coordinate the coupled and decoupled agents, we design a reward-driven scheduler, as shown in Figure 2 ❹, to dynamically select the most effective action proposed by the two agents.

During the cleaning process, the coupled agent $\mathcal{A}_c$ and the decoupled agent $\mathcal{A}_{d+r}$ operate in parallel within each window. Since they explore complementary cleaning paradigms, the two agents provide heterogeneous candidate actions for the current window. Given the current state, each agent recommends one candidate action according to its learned policy, and the scheduler selects the action with the highest normalized reward to update the data state.

For the coupled cleaning agent $\mathcal{A}_c$, the effectiveness of a coupled action a on state s is measured by the reward r, which reflects the distance change Δ from the data state before and after applying the action to the target reference representation s^{ref} (details in §5). The distances before and after the action are denoted as $d_w(s) = \sum_{m=1}^{M} \sum_{n=t}^{t+l-1} |s_m^{ref}(t_n) - s_m(t_n)|$, and $d_w(s') = \sum_{m=1}^{M} \sum_{n=t}^{t+l-1} |s_m^{ref}(t_n) - s'_m(t_n)|$, respectively. Thus, the distance change is defined as $\Delta = d_w(s') - d_w(s)$, and the r is computed as:

$$r = R(s, a, s') = \begin{cases} \log(-\Delta + 1) & \text{if } \Delta < 0 \\ 0 & \text{if } \Delta = 0 \\ -\log(\Delta + 1) & \text{if } \Delta > 0 \end{cases} \quad (1)$$

Therefore, actions that reduce the discrepancy to the reference representation receive positive rewards, while those that increase the discrepancy are penalized.

In contrast, for the decoupled cleaning agent $\mathcal{A}_{d+r}$, the effectiveness of the detection action is evaluated by the detection reward dr, which measures the detected error locations against reference-derived error positions using F1 score:

$$dr = R(ds, da, ds') = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}} \quad (2)$$

where error positions are identified by comparing the observed state with the target reference representation during training or the estimated reference during inference; Precision $= \frac{TP}{TP+FP}$ and Recall $= \frac{TP}{TP+FN}$, with TP, FP, and FN denoting correctly detected erroneous data, clean data incorrectly identified as erroneous, and undetected erroneous data, respectively. This detection reward formulation balances omission and commission errors, encouraging detection strategies that achieve high coverage and accuracy.

The effectiveness of the repair action is evaluated using the repair reward rr, which follows the same distance-change design as in Eq (1), encouraging repair actions that reduce data contamination and penalizing those that degrade data quality.

Based on these rewards, the scheduler dynamically selects an agent at each decision step. To accommodate different cleaning paradigms, the detection and repair rewards of the decoupled agent are aggregated into $dr + rr$ to enable comparison with the coupled agent's reward r. The r and $dr + rr$ are then independently normalized using min-max normalization based on their respective reward histories, each of which maintains up to 200 raw reward observations. If a history is unavailable, a sigmoid transformation is used as a fallback. The scheduler executes a if the coupled agent's normalized reward r is higher, and (da, ra) otherwise, to update the current window state. By dynamically switching between coupled and decoupled cleaning paradigms based on window characteristics, the scheduler leverages their complementary strengths to improve robustness against heterogeneous and evolving errors.

5 Discriminator

As the Cleaning Path refines contaminated data within each window, it needs to determine whether to continue cleaning or proceed to the next window. However, different windows may exhibit diverse error distributions and contamination levels, making a fixed cleaning depth (i.e., the number of cleaning iterations) across all windows unsuitable. Insufficient iterations may leave residual errors, whereas excessive iterations can introduce unnecessary modifications and additional computational costs. To enable adaptive termination, we introduce a multi-evidence Discriminator that dynamically determines the termination point for each window, as illustrated in Figure 3. Specifically, the Discriminator implements the termination criterion Γ by integrating multiple evidence signals from the cleaning process and supports two termination modes.

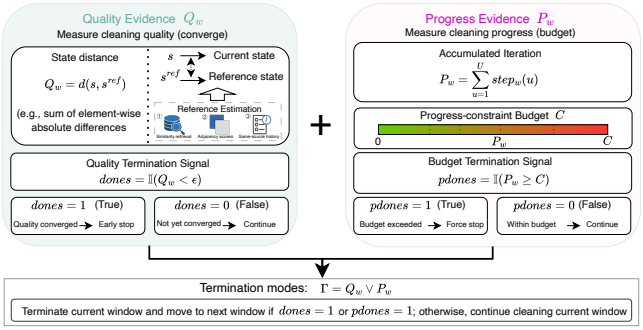


Fig. 3: Discriminator based on multi-evidence signals

5.1 Multiple evidence signals

To mitigate residual errors and over-cleaning, the Discriminator introduces multiple evidence signals to obtain state information, since a single termination signal cannot simultaneously characterize data quality and cleaning progress. Specifically, it combines two complementary evidences, namely *quality evidence* and *progress evidence*, denoted as $\Gamma = \mathbb{1}(Q_w < \epsilon) \vee \mathbb{1}(P_w > C)$, i.e., $\Gamma = dones \vee pdones$.

Quality evidence Q_w. Quality evidence measures whether the current cleaning state has converged to a clean distribution. The Discriminator estimates the remaining deviation by computing the distance between the current state s and a reference s^{ref}:

$$Q_w = d(s, s^{ref}) \quad (3)$$

where $d(\cdot)$ denotes the state distance function, such as the element-wise sum of absolute differences. The quality evidence is then converted into a binary termination signal: $dones = \mathbb{I}(Q_w < \epsilon)$, where

$\mathbb{I}(\cdot)$ denotes the indicator function, and ϵ is a predefined threshold. When $Q_w < \epsilon$, the current state is considered sufficiently clean and the scheduling terminates; otherwise, further cleaning operations are required.

While the ground-truth clean data serve as s^{ref} during training, they are unavailable during inference. We therefore employ a hierarchical reference estimation mechanism to approximate the target clean distribution at inference time. Specifically, the Discriminator first retrieves the most similar historical cleaned window by computing cosine similarities between the flattened current window and historical cleaned windows, with similarity scores mapped to ([0,1]). A match is accepted only if its similarity exceeds an adaptive threshold ξ, initialized to 0.85 and adjusted every five windows based on the mean and variance of recent cleaning rewards: ξ increases when rewards are high or stable and decreases when rewards are low or highly variable, remaining within ([0.6,0.95]). If no reliable match is found, the most recently cleaned window is used as a fallback; for the initial window, an externally provided neighboring window from the same data source serves as the reference until enough historical cleaned windows have accumulated for retrieval. Furthermore, when reference data are unavailable (i.e., additional data from the same source or historical cleaning results), we provide a *No-reference* setting, as detailed in §8.4.4. We empirically evaluate the effectiveness of the Reference and *No-reference* settings in supporting quality assessment and automatic termination (§8.4.4).

Progress evidence P_w. Progress evidence P_w determines whether additional scheduling steps remain beneficial. Beyond quality convergence, the Discriminator monitors the cleaning progress to prevent both over-cleaning and insufficient cleaning. Under complex error patterns or during early training, the scheduling process may enter low-benefit iterations, leading to redundant computation and potential over-cleaning. Conversely, insufficient scheduling may leave some errors unresolved.

To address this issue, we define the cumulative number of executed iterations within the current window as the progress evidence: $P_w = \sum_{u=1}^{U} step_w(u)$, where $step_w(u)$ denotes the iteration step of the u-th cleaning operation. Subsequently, the progress evidence is converted into a binary progress termination indicator $pdones \in \{0, 1\}$: $pdones = \mathbb{I}(P_w \geq C)$, where C represents the maximum iteration budget for each window. When the cumulative number of iterations exceeds the predefined budget C, i.e., $pdones = 1$, the Discriminator forcibly terminates the cleaning process of the current window, even if the quality convergence condition has not been fully satisfied. Otherwise, $pdones = 0$, indicating that the current window remains within the allocated budget and can continue subsequent cleaning operations.

In summary, multiple evidence signals jointly consider cleaning effectiveness and resource consumption, enabling quality-driven and budget-driven termination.

5.2 Termination modes

Based on the above evidence, the Discriminator supports two complementary termination modes. The first is *quality-driven termination*, where the current state converges to satisfactory cleaning quality before the maximum iteration is exhausted, indicating that further operations are unnecessary. The second is *budget-driven termination*, where the maximum iteration C is reached before sufficient quality convergence is achieved. In this case, the Discriminator proactively terminates the current window to avoid excessive cost, potential over-cleaning, and missed-error risks. The combination of two modes enables the cleaning process to maintain dynamic nature and computational stability across windows with different contamination characteristics and cleaning difficulties.

Overall, the Discriminator adapts cleaning depth to each window—terminating easy windows early while granting difficult windows more iterations—balancing cleaning effectiveness against missed errors, over-cleaning, and redundant computation.

Properties 1. *The Discriminator bounds the scheduling steps per window by C via $pdones = \mathbb{I}(P_w \geq C)$, yielding at most $N_w C$ operator executions (or $2N_w C$ if both agents execute operators before comparison). For quality-terminated windows, $dones = \mathbb{I}(Q_w < \epsilon)$, where $Q_w = d(s, s^{ref})$, ensures $d(s, s^{ref}) < \epsilon$; this guarantee does not hold for budget-terminated windows. Moreover, by the triangle inequality, $d(s, D_w^*) \leq \epsilon + \delta_r$, where $\delta_r = d(s^{ref}, D_w^*)$, D_w^* is the ground-truth clean window and δ_r quantifies the reference error. Thus, the error bound depends on reference quality and may become less informative for non-stationary time series. The performance degradation with cross-dataset references in §8.4.4 illustrates this limitation.*

6 Adaptation mechanism

This section presents the generalization of Cleaner for unseen datasets, error types and TSC methods via *Adaptation Mechanism*. As shown in Figure 4, it consists of *Knowledge Integration* and *Model Adaptation*, which distills transferable cleaning knowledge from multiple datasets and enables lightweight adaptation to new data.

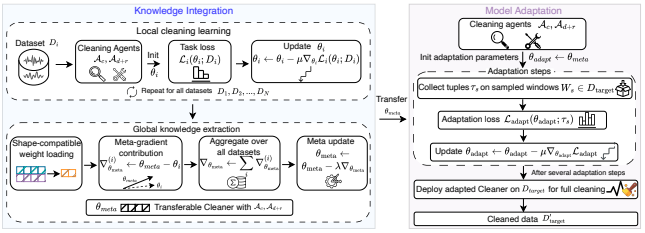


Fig. 4: Adaptation mechanism

6.1 Knowledge integration

To achieve cross-domain generalization for autonomously scheduled cleaning, the *Knowledge Integration* extracts transferable cleaning knowledge across multiple datasets via meta-learning. Specifically, the coupled cleaning agent $\mathcal{A}_c$ and decoupled cleaning agent $\mathcal{A}_{d+r}$ are trained over multiple domains $D_1, D_2, \ldots, D_N$ (e.g., energy, power load, and industrial sensor data) to learn generalized cleaning policies under diverse distributions. As presented in Alg. 1, the overall process consists of two stages: local cleaning learning and global knowledge extraction. The former learns dataset-specific cleaning behaviors, while the latter aggregates task knowledge to update shared meta-parameters θ_{meta}.

6.1.1 Local cleaning learning. In this stage, each dataset D_i is independently processed to learn dataset-specific cleaning behaviors. The agents $\mathcal{A}_c$ and $\mathcal{A}_{d+r}$ are initialized with task parameters

Algorithm 1 Knowledge integration

Input: A set of datasets $\{D_1, D_2, \ldots, D_N\}$, learning rates μ, λ
Output: Meta parameters θ_{meta}

/* **Local cleaning learning** */
1: **for** each dataset D_i **do**
2: Create $\mathcal{A}_c$ and $\mathcal{A}_{d+r}$
3: Initialize task parameters θ_i
4: Compute task-specific loss $\mathcal{L}_i(\theta_i; D_i)$
5: Update $\theta_i \leftarrow \theta_i - \mu \nabla_{\theta_i} \mathcal{L}_i(\theta_i; D_i)$

/* **Global knowledge extraction** */
6: Load shape-compatible weights from θ_i
7: Compute meta-gradient contribution $\nabla^{(i)}_{\theta_{\text{meta}}} \leftarrow \theta_{\text{meta}} - \theta_i$
8: Accumulate meta-gradient $\nabla_{\theta_{\text{meta}}} \leftarrow \sum_i \nabla^{(i)}_{\theta_{\text{meta}}}$
9: Update meta parameters $\theta_{\text{meta}} \leftarrow \theta_{\text{meta}} - \lambda \nabla_{\theta_{\text{meta}}}$
10: Return transferable Cleaner with $(\mathcal{A}_c, \mathcal{A}_{d+r}) \leftarrow \theta_{\text{meta}}$

Algorithm 2 Model adaptation

Input: Target dataset D_{target}, meta parameters θ_{meta}, learning rate μ
Output: Cleaned data D'_{target}

1: Create $\mathcal{A}_c, \mathcal{A}_{d+r}$
2: Initialize adaptation parameters $\theta_{\text{adapt}} \leftarrow \theta_{\text{meta}}$
3: **for** each adaptation step **do**
4: Sample adaptation windows $W_s \subset D_{\text{target}}$
5: Apply $(\mathcal{A}_c, \mathcal{A}_{d+r})$ on W_s to obtain tuples τ_s
6: Compute adaptation loss $\mathcal{L}_{\text{adapt}}(\theta_{\text{adapt}}; \tau_s)$
7: Update $\theta_{\text{adapt}} \leftarrow \theta_{\text{adapt}} - \mu \nabla_{\theta_{\text{adapt}}} \mathcal{L}_{\text{adapt}}$
8: Deploy the adapted Cleaner on D_{target} for full cleaning
9: Return the entire cleaned data D'_{target}

θ_i and optimized by minimizing the task-specific loss $\mathcal{L}_i(\theta_i; D_i)$: $\theta_i \leftarrow \theta_i - \mu \nabla_{\theta_i} \mathcal{L}_i(\theta_i; D_i)$. This step captures the dataset characteristics, including variable correlations, temporal dependencies, and error patterns within the dataset, without cross-dataset interaction.

6.1.2 Global knowledge extraction. After local learning, the task parameters $\{\theta_i\}_{i=1}^N$ are aggregated to update the shared initialization θ_{meta}. To handle different parameter dimensions, a shape-compatible weight-loading mechanism is applied before meta-gradient computation. Specifically, parameters are reused when their tensor shapes are compatible; otherwise, incompatible dimensions are aligned through channel-wise truncation or zero-padding. For example, a TCN layer with weight dimensions $(64, 10, 3)$ in an energy dataset and $(64, 5, 3)$ in a power load dataset can still share the 64 output channels and kernel size 3, while adjusting the input channels. Meanwhile, input and output layers are independently initialized from meta optimization.

This mechanism enables flexible parameter sharing while preserving stable feature extraction across datasets. Then, task-specific meta-gradient contributions $\nabla^{(i)}_{\theta_{\text{meta}}}$ are computed as the difference between θ_{meta} and θ_i, aggregated across datasets, and used to update θ_{meta} (Alg. 1, lines 7-9). Through meta-training $\theta_{\text{meta}} \leftarrow \theta_{\text{meta}} - \lambda \nabla_{\theta_{\text{meta}}}$ [32], θ_{meta} captures transferable cleaning knowledge and provides an effective initialization for Cleaner with $\mathcal{A}_c$ and $\mathcal{A}_{d+r}$, enabling efficient adaptation to unseen datasets.

6.2 Model adaptation

Although Cleaner acquires transferable knowledge via *Knowledge Integration*, target datasets may still exhibit unseen error types, variable dimensions, or distributional characteristics. To address this issue, the *Model Adaptation* refines $\mathcal{A}_c$ and $\mathcal{A}_{d+r}$ on the target dataset with limited interactions, as presented in Alg. 2.

Given a target dataset D_{target}, both $\mathcal{A}_c$ and $\mathcal{A}_{d+r}$ are initialized with the meta-learned parameters θ_{meta}, which serve as the starting point for target-specific adaptation. The agents then interact with sampled windows from D_{target} to collect transition tuples τ_s and optimize θ_{adapt} through gradient-based updates: $\theta_{\text{adapt}} \leftarrow \theta_{\text{adapt}} - \mu \nabla_{\theta_{\text{adapt}}} \mathcal{L}_{\text{adapt}}(\theta_{\text{adapt}}; \tau_s)$. Here, θ_{adapt} is evolved from the initial θ_{meta} via such interactions and encapsulates the separately updated parameters of both $\mathcal{A}_c$ and $\mathcal{A}_{d+r}$. This process enables the agents to adjust their cleaning policies according to the target dataset's statistical characteristics and error patterns. After adaptation, Cleaner with the adapted parameters θ_{adapt} are deployed for full cleaning on D_{target}, yielding the cleaned data D'_{target}.

Furthermore, users can extend Cleaner by adding new cleaning methods to its tool library, enabling the system to adapt its cleaning policies to these methods.

Together, model adaptation and knowledge integration form the Adaptation Mechanism that supports transferability and target-specific refinement for robust autonomous scheduled cleaning. The effectiveness of adaptation is empirically studied in §8.6.

7 Complexity analysis

We analyze the time and space complexities of Cleaner, where T, M, and l denote the time-series length, number of variables, and window size, respectively. Let C be the scheduling budget (i.e., the maximum number of cleaning iterations per window), Φ_{net} the TVACN inference cost per decision step, and Φ_c, Φ_d, and Φ_r the execution costs of coupled cleaning, detection, and repair actions, respectively.

At each step, Cleaner evaluates only two candidate actions: coupled cleaning and decoupled detection-and-repair. This design eliminates exhaustive enumeration over operator combinations: the exhaustive search space is on the order of $\sum_{k=1}^{n} P(n,k)$ (as described in §1), whereas our RL formulation restricts each step to merely two candidates, reducing the number of operator executions per window from combinatorial explosion to $O(C)$. Accordingly, the upper-bound computational cost for a single window is $\Phi_w = C\big(\Phi_{\text{net}} + \max(\Phi_c, \Phi_d + \Phi_r) + lM\big)$, where lM represents the reference-related cost. The overall time complexity is therefore $O\big(N_w \Phi_w\big)$, with $N_w = \lceil T/l \rceil$ denotes the number of windows. Hence, the scheduling overhead scales linearly with N_w and C, without requiring enumeration of the combinatorial search space.

The space complexity during training is $O\big(\mathcal{P}_{\text{net}} + (\mathcal{B}_c + \mathcal{B}_d + \mathcal{B}_r + H)lM\big)$, where $\mathcal{P}_{\text{net}}$ denotes the TVACN parameter storage, $\mathcal{B}_c$, $\mathcal{B}_d$, and $\mathcal{B}_r$ are the replay buffer sizes of the coupled, detection, and repair agents, respectively, and H is the cache size of historical reference windows. During inference, replay buffers are removed, reducing the space complexity to $O(\mathcal{P}_{\text{net}} + HlM)$.

8 Experiment

8.1 Experiment settings

8.1.1 Datasets. We evaluate Cleaner on six datasets: OilTemp_real [7], Power_real [7], Energy_real [7], IDF [29], OilTemp [7], and Power [7].

Table 4: Characteristics of datasets

Datasets	Description	Error Sources	Size	Error types
OilTemp_real	Wind turbine lubrication-system temperatures	Real	10000 * 4	NS, DF, VA, AC
Power_real	Wind power generation operations	Real	11200 * 5	DF, VA, AC
Energy_real	Low-energy building measurements	Real	9000 * 4	DF, GC, AC
IDF	Power-plant induced draft fan operations	Synthetic	20000 * 38	NS, DF, VA, GC, AC
OilTemp	Wind turbine lubrication-system temperatures	Synthetic	10000 * 4	NS, DF, VA, GC, AC
Power	Wind power generation operations	Synthetic	11200 * 5	NS, DF, VA, GC, AC

The first three are real-world datasets, and the last three are synthetic. Following prior studies [10, 12, 52], we construct the synthetic datasets by injecting five error types–AC, GC, DF, VA, and NS–into clean data segments, as detailed in §2. The clean segments are publicly available from the corresponding references and serve as the ground truth for both the real-world and synthetic datasets with injected errors. In our experiments, the N-1 datasets are used to construct training tasks by injecting errors online into clean data, while the N-th dataset is reserved for testing and contains either real-world errors or different synthetic errors. Unless otherwise stated, the error rate is set to $e = 10\%$, i.e., 10% of timestamps are corrupted. We study the impacts of error rate and the error type in §8.5. Table 4 summarizes the characteristics of datasets.

8.1.2 Baselines. For an extensive evaluation, we compare Cleaner with 24 applicable baselines and variants in Table 5. They include state-of-the-art **single-error** and **multi-error** methods, where the former are designed for a single error type and the latter handles multiple error types. We further include **ensemble** baselines that select a fixed combination of methods globally for all windows (OneClean) or adaptively choose the optimal method for each window (BestClean), as well as an AllClean that iteratively applies all methods, to evaluate the benefit of method integration in Cleaner. In addition, we compare **optimization-based** baselines, including conventional optimization algorithms and RL-based multi-objective optimization methods, to assess the RL scheduling mechanism used in Cleaner. We also implement two **Cleaner variants**, Cleaner-MAB and Cleaner-DQN, which replace our TVACN (§4.3) with a Multi-armed bandit and DQN respectively, to evaluate the contribution of TVACN to method scheduling in Cleaning Path (§8.7). During training, Cleaner uses ground-truth information to compute rewards for evaluating cleaning actions, while during inference, it employs hierarchical reference estimation. Regarding training signals, OneClean and BestClean in the Ensemble category, along with all Optimization-based baselines except DemandClean, use the same ground-truth-based reward computation as Cleaner. In contrast, DemandClean uses error labels and downstream task feedback to guide its cleaning decisions. While Single-error and Multi-error methods generally rely on clean data or predefined constraints.

Implementation and Environment. Cleaner and all baselines are implemented in Python and evaluated on an Ubuntu machine with 3.6GHz Intel Xeon Gold 5122 CPU, 128GB RAM, and 12GB GPU. All reported results are averaged over three runs, with standard deviations reported for primary metrics.

8.1.3 Metrics. The literature and practice have introduced diverse metrics to assess the cleaning performance, primarily focusing three aspects: effectiveness, efficiency and tasks. Specifically, for detection effectiveness, we use Precision & Recall & F1, where $P = \frac{TP}{TP+FP}$, $R = \frac{TP}{TP+FN}$, and F1 is the harmonic mean of P and R, as detailed in §4.6. For repair effectiveness, we adopt (1) Mean Squared Error: $MSE = \sum_{m=1}^{M}\sum_{n=1}^{T}\frac{(D'_m[t_n]-D^*_m[t_n])^2}{M\cdot T}$. (2) Mean Normalized Absolute Distance: $MNAD = \sum_{m=1}^{M}\sum_{n=1}^{T}\frac{|Norm(D'_m[t_n]-D^*_m[t_n])|}{M\cdot T}$, where $Norm$ denotes normalization. (3) Relative Repair Accuracy: $RRA = 1-\frac{\triangle(D',D^*)}{\triangle(D,D^*)+\triangle(D,D')}$. (4) Mean Absolute Error: $MAE = \sum_{m=1}^{M}\sum_{n=1}^{T}\frac{|D'_m[t_n]-D^*_m[t_n]|}{M\cdot T}$ [7, 10]. For efficiency, we report inference time to measure the time cost. For tasks, we use Silhouette Score: $SS = \frac{\sum_{m=1}^{M}\sum_{n=1}^{T}\frac{b(D'_m[t_n])-a(D'_m[t_n])}{max(a(D'_m[t_n]),b(D'_m[t_n]))}}{M\cdot T}$, where a and b denote the average intra- and inter-cluster distances.

Table 5: Summary of cleaning baselines and variants

Category	Baseline Methods	Description
Single-error	1) Vari[48], 2) Speed(L)[42] 3)Speed(G)[42], 4)Speed+Acc(L)[39] 5) Speed+Acc(G)[39]	Employ constraints on columns
	6) MTSClean-Row[10]	Use constraints on rows
	7) EWMA[17]	Exponentially weighted moving average
	8) IMR[51]	Label-guided minimal repair
Multi-error	9) Clean4MTS [7], 10) Clean4MTS+ [7], 11) MTSClean [10]	Combine row and column constraints
	12) Akane [22]	Cost-constrained perplexity minimization
	13) MEDetect [11]	Hierarchical VAE-based detection
	14) L2C [24]	SVR-ARIMA-Loess Integration
Ensemble	15) OneClean	Globally select a combination
	16) AllClean	Iteratively run all methods
	17) BestClean	Select the best per window
Optimization-based	18) GAClean	Genetic algorithm [44]
	19) PSOClean	Particle swarm optimization [47]
	20) ACOClean	Ant colony optimization [15]
	21) DEClean	Differential evolution [43]
	22) DemandClean [36]	RL multi-objective
Cleaner variants	23) Cleaner-MAB	Multi-armed bandit [38]
	24) Cleaner-DQN	DQN-based RL [35]

8.2 Comparison with baselines

8.2.1 Effectiveness. We evaluate the overall cleaning effectiveness of Cleaner against all baselines on six datasets. Tables 6 and 7 reports the results on real-world and synthetic errors. Cleaner achieves consistently strong performance across both settings, obtaining the best or comparable results on most datasets in terms of different metrics. Although some baselines outperform Cleaner on individual metrics, their advantages are not consistent across datasets. Compared with BestClean, Cleaner achieves comparable performance and even outperforms it on some metrics, further demonstrating its superiority.

8.2.2 Efficiency. We compare the inference runtime and per-window inference latency of Cleaner with the main competing baselines in Table 8 and Figure 5. Table 8 shows that Cleaner achieves competitive inference efficiency. Its overhead primarily stems from the Scheduler in the Cleaning Path, where both the coupled and decoupled cleaning agents must execute their actions and compute rewards before selecting. Moreover, the dynamic scheduling process requires maintaining and updating the network architecture. Figure 5 further presents the per-window inference latency over three real-world datasets, showing that Cleaner achieves reasonable per-window latency and is comparable to ACOClean on all three datasets. To further reduce inference overhead, we evaluate two lightweight variants in §8.3, which achieve speedups of approximately 3.61× and 13.20×, respectively, over the average inference runtime of the baselines on the IDF dataset.

Table 6: The effectiveness of Cleaner on real errors

Methods	Power_real							OilTemp_real							Energy_real						
	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓
Vari	3.12e-3	7.58e-3	0.658	0.509	0.657	0.573	2.96e-3	3.19e-4	3.43e-3	0.732	0.969	0.601	0.741	2.03e-3	3.04e-3	1.02e-2	0.722	0.346	0.999	0.515	6.69e-3
Speed(L)	4.94e-3	7.72e-3	0.586	0.647	0.582	0.613	4.82e-3	1.13e-3	6.16e-3	0.456	0.718	0.329	0.451	4.17e-3	4.46e-3	1.49e-2	0.622	0.247	0.999	0.396	7.75e-3
Speed(G)	2.02e-3	4.66e-3	0.741	0.995	0.586	0.738	3.59e-3	5.97e-4	4.17e-3	0.608	0.997	0.381	0.551	3.48e-3	1.56e-3	7.53e-3	0.800	0.335	0.999	0.501	4.91e-3
Speed+Acc(L)	6.35e-2	2.63e-2	0.137	0.171	0.282	0.213	1.01e-2	4.41e-3	1.28e-2	0.093	0.102	0.081	0.090	6.47e-3	1.65e-2	2.74e-2	0.438	0.182	0.765	0.294	1.08e-2
Speed+Acc(G)	2.43e-3	5.11e-3	0.717	0.934	0.586	0.721	3.78e-3	6.48e-4	4.39e-3	0.589	0.958	0.366	0.529	3.57e-3	2.08e-3	7.98e-3	0.786	0.786	0.346	0.481	5.27e-3
MTSClean-Row	2.55e-3	4.24e-3	0.740	0.998	0.579	0.732	4.24e-3	2.28e-3	6.31e-3	0.228	0.441	0.063	0.110	5.80e-3	1.43e-2	1.63e-2	0.325	0.978	0.217	0.356	1.61e-2
EWMA	1.79e-2	1.72e-2	0.150	0.500	0.144	0.224	8.77e-3	1.92e-3	1.22e-2	0.174	0.560	0.125	0.204	5.46e-3	1.57e-2	2.77e-2	0.234	0.024	0.980	0.047	1.58e-2
IMR	1.58e-2	7.68e-3	0.402	0.998	0.239	0.385	7.68e-3	1.84e-3	5.23e-3	0.531	0.726	0.516	0.603	3.82e-3	1.62e-2	1.59e-2	0.339	**0.999**	0.206	0.342	1.59e-2
Clean4MTS	1.61e-3	1.75e-3	0.909	0.820	0.934	0.873	1.67e-3	6.80e-4	1.97e-3	0.840	0.998	**0.833**	**0.908**	1.95e-3	4.71e-3	5.22e-3	0.853	0.854	0.871	0.863	5.05e-3
Clean4MTS+	1.27e-3	1.55e-3	0.921	0.735	**0.947**	0.828	1.46e-3	6.82e-4	1.99e-3	0.838	0.997	0.822	0.901	1.96e-3	4.24e-3	4.84e-3	0.865	0.835	0.890	0.861	4.69e-3
MTSClean	2.48e-3	4.01e-3	0.872	0.998	0.586	0.738	2.32e-3	2.40e-3	6.65e-3	0.039	0.384	0.005	0.010	6.59e-3	1.78e-2	1.82e-2	0.164	**0.999**	0.109	0.196	1.82e-2
Akane	3.34e-3	3.42e-3	0.760	0.986	0.342	0.508	3.31e-3	1.82e-3	5.18e-3	0.083	0.896	0.026	0.050	5.15e-3	1.43e-2	8.47e-2	0.242	0.218	**1.000**	0.358	**2.89e-3**
MEDetect	1.29e-2	8.19e-3	0.374	0.971	0.248	0.396	7.82e-3	2.05e-3	6.44e-3	0.115	**1.000**	0.045	0.086	6.31e-3	1.06e-2	1.50e-2	0.441	0.978	0.429	0.597	1.41e-2
OneClean	5.66e-3	7.00e-3	0.618	0.624	0.502	0.556	4.61e-3	2.04e-3	7.22e-3	0.288	0.352	0.111	0.168	5.27e-3	6.32e-3	1.28e-2	0.600	0.776	0.576	0.661	1.03e-2
AllClean	**1.00e-3**	5.79e-3	0.751	0.650	0.625	0.637	1.55e-3	5.63e-4	5.37e-3	0.643	0.837	0.583	0.687	1.92e-3	3.49e-3	1.65e-2	0.629	0.725	0.910	0.807	5.99e-3
BestClean	1.15e-3	2.52e-3	0.860	**1.000**	0.635	0.776	2.49e-3	2.30e-4	2.27e-3	0.807	0.982	0.619	0.759	1.96e-3	3.04e-3	7.26e-3	0.781	0.998	0.852	0.919	7.00e-3
GAClean	2.91e-3	6.16e-3	0.709	0.668	0.647	0.657	2.75e-3	9.30e-4	5.43e-3	0.577	0.807	0.471	0.595	3.02e-3	5.51e-3	1.15e-2	0.668	0.803	0.712	0.755	8.37e-3
PSOClean	3.37e-3	6.00e-3	0.701	0.698	0.628	0.661	3.22e-3	8.57e-4	5.70e-3	0.589	0.769	0.513	0.616	2.66e-3	2.32e-3	8.76e-3	0.770	0.824	0.927	0.873	5.28e-3
ACOClean	5.41e-3	6.80e-3	0.685	0.743	0.582	0.653	2.86e-3	6.26e-4	4.02e-3	0.687	0.880	0.525	0.657	2.32e-3	4.56e-3	1.13e-2	0.673	0.811	0.736	0.772	8.28e-3
DEClean	1.20e-3	5.82e-3	0.756	0.654	0.707	0.679	**1.23e-3**	2.14e-4	2.29e-3	0.817	0.975	0.606	0.748	1.62e-3	2.26e-3	8.61e-3	0.774	0.862	0.885	0.873	5.22e-3
DemandClean	2.27e-2	8.87e-3	0.534	0.472	0.287	0.357	8.87e-3	9.16e-1	8.56e-2	0.058	0.186	0.108	0.137	8.56e-2	5.81e-2	1.80e-2	0.350	0.480	0.341	0.399	1.80e-2
Cleaner	1.01e-3	**1.49e-3**	**0.931**	0.903	0.897	**0.899**	1.32e-3	**1.98e-4**	**1.56e-3**	**0.895**	0.976	0.751	0.848	**1.45e-3**	**1.33e-3**	**4.42e-3**	**0.874**	0.959	0.890	**0.922**	4.20e-3

Table 7: The effectiveness of Cleaner on synthetic errors

Methods	IDF							OilTemp							Power						
	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓
Vari	2.08e-5	3.29e-4	0.585	0.814	**0.829**	0.845	9.05e-5	1.24e-4	1.57e-3	0.669	0.981	0.689	0.810	1.10e-3	2.45e-4	2.26e-3	0.461	0.607	0.513	0.555	1.22e-3
Speed(L)	6.37e-5	4.21e-4	0.199	0.398	0.098	0.157	2.67e-4	3.66e-4	2.69e-3	0.397	0.554	0.193	0.286	1.89e-3	3.46e-4	2.35e-3	0.274	0.483	0.123	0.196	1.77e-3
Speed(G)	3.56e-5	2.22e-4	0.504	0.802	0.408	0.541	1.83e-4	3.09e-4	2.17e-3	0.466	0.814	0.281	0.417	1.78e-3	3.42e-4	2.22e-3	0.251	0.703	0.111	0.191	1.82e-3
Speed+Acc(L)	6.73e-5	3.09e-4	0.181	0.335	0.097	0.150	2.86e-4	7.31e-4	3.72e-3	0.199	0.144	0.075	0.099	2.48e-3	4.47e-4	2.45e-3	0.171	0.397	0.066	0.113	2.01e-3
Speed+Acc(G)	3.56e-5	2.22e-4	0.504	0.802	0.408	0.541	1.83e-4	3.06e-4	2.16e-3	0.473	0.787	0.286	0.419	1.75e-3	3.36e-4	2.20e-3	0.269	0.696	0.117	0.200	1.79e-3
MTSClean-Row	5.52e-5	2.75e-4	0.330	0.604	0.240	0.343	2.26e-4	4.14e-4	2.31e-3	0.280	0.856	0.222	0.352	2.26e-3	4.25e-4	2.11e-3	0.069	**0.999**	0.029	0.056	2.09e-3
EWMA	1.02e-4	3.35e-3	0.049	0.334	0.182	0.236	6.05e-4	5.03e-4	8.23e-3	0.156	0.610	0.126	0.208	2.02e-3	4.23e-4	9.03e-3	0.122	0.683	0.151	0.247	1.66e-3
IMR	4.47e-5	2.18e-4	0.497	0.642	0.567	0.602	1.94e-4	3.42e-4	1.85e-3	0.559	0.804	0.428	0.558	1.58e-3	2.93e-4	1.71e-3	0.506	0.733	0.430	0.542	1.38e-3
Clean4MTS	6.51e-5	2.97e-4	0.006	0.375	0.004	0.008	2.92e-4	2.26e-4	1.61e-3	0.589	0.989	0.481	0.647	1.59e-3	3.26e-4	1.68e-3	0.400	0.942	0.295	0.449	1.64e-3
Clean4MTS+	6.51e-5	2.97e-4	0.006	0.333	0.004	0.008	2.92e-4	2.38e-4	1.65e-3	0.575	0.988	0.465	0.632	1.63e-3	3.04e-4	1.62e-3	0.437	0.935	0.322	0.479	1.57e-3
MTSClean	6.17e-5	2.89e-4	0.083	0.609	0.047	0.087	2.80e-4	5.67e-4	2.67e-3	0.016	0.999	0.003	0.005	2.67e-3	4.39e-4	2.19e-3	0.057	0.289	0.029	0.053	2.10e-3
Akane	3.66e-4	2.13e-3	0.037	0.027	0.075	0.040	5.12e-4	4.48e-4	2.13e-3	0.032	**1.000**	0.013	0.026	2.11e-3	5.78e-4	3.71e-3	0.153	0.210	0.189	0.199	1.56e-3
MEDetect	1.20e-4	5.97e-4	0.010	0.111	0.001	0.002	5.51e-4	5.72e-4	2.78e-3	0.013	0.080	0.020	0.032	2.67e-3	4.67e-4	2.38e-3	0.220	0.010	0.050	0.017	2.17e-3
OneClean	5.54e-5	3.87e-4	0.582	0.818	0.504	0.624	2.92e-4	3.14e-4	2.21e-3	0.452	0.791	0.344	0.480	1.85e-3	3.00e-4	1.91e-3	0.422	0.614	0.362	0.456	1.51e-3
AllClean	4.30e-5	6.84e-4	0.536	0.745	0.791	0.768	1.62e-4	2.17e-4	3.15e-3	0.512	0.853	0.668	0.749	1.08e-3	2.49e-4	2.77e-3	0.446	0.613	0.608	0.610	**1.08e-3**
BestClean	2.77e-5	2.40e-4	**0.737**	**0.991**	0.690	0.813	2.16e-4	1.31e-4	**1.28e-3**	**0.704**	0.980	0.681	0.804	1.16e-3	**1.16e-4**	**1.13e-3**	**0.654**	0.978	0.614	**0.755**	1.09e-3
GAClean	4.91e-5	5.75e-4	0.550	0.755	0.690	0.721	2.10e-4	2.37e-4	1.78e-3	0.574	0.924	0.459	0.614	1.51e-3	3.11e-4	1.92e-3	0.370	0.730	0.281	0.406	1.62e-3
PSOClean	4.48e-5	5.65e-4	0.560	0.781	0.711	0.745	2.01e-4	2.42e-4	2.43e-3	0.538	0.803	0.558	0.659	1.30e-3	3.24e-4	2.70e-3	0.392	0.471	0.479	0.475	1.32e-3
ACOClean	4.66e-5	5.15e-4	0.575	0.799	0.681	0.736	2.14e-4	2.61e-4	2.08e-3	0.517	0.810	0.422	0.555	1.61e-3	3.36e-4	2.56e-3	0.359	0.467	0.344	0.396	1.48e-3
DEClean	4.58e-5	5.01e-4	0.576	0.814	0.655	0.726	2.18e-4	1.27e-4	1.57e-3	0.681	0.981	0.697	0.815	1.01e-3	1.97e-4	1.77e-3	0.522	0.793	0.554	0.652	1.21e-3
DemandClean	2.22e-1	1.62e-2	0.029	0.141	0.182	0.159	1.62e-2	6.72e-1	6.12e-2	0.031	0.133	0.184	0.154	6.12e-2	3.47e-2	1.38e-2	0.091	0.112	0.131	0.121	1.38e-2
Cleaner	**1.51e-5**	**1.36e-4**	0.720	0.949	0.801	**0.868**	**8.74e-5**	**1.11e-4**	1.59e-3	0.701	0.971	**0.717**	**0.825**	**9.40e-4**	1.63e-4	1.59e-3	0.645	0.854	**0.632**	0.726	1.11e-3

Table 8: The inference runtime of Cleaner (seconds)

Methods	IDF	OilTemp	Power	Power_real	OilTemp_real	Energy_real
GAClean	7322.683	22.414	**24.546**	**20.968**	**20.622**	24.265
PSOClean	6795.091	20.947	32.297	21.447	22.671	20.405
ACOClean	7580.942	**20.835**	29.148	59.996	35.972	20.388
DEClean	**6634.587**	22.657	27.939	23.733	21.445	**9.091**
DemandClean	66276.417	719.659	1502.697	477.703	943.191	206.856
Cleaner	6999.102	94.284	143.787	60.846	41.723	21.157

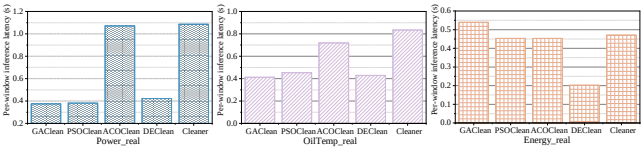


Fig. 5: The per-window inference latency (seconds)

Table 9: The effectiveness-efficiency tradeoff of Cleaner

Forms	IDF							
	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓	Time (s) ↓
Cleaner	**1.51e-5**	**1.36e-4**	**0.720**	**0.949**	**0.801**	**0.868**	**8.74e-5**	6999.102
Disable meta learning	2.87e-5	3.63e-4	0.682	0.918	0.754	0.838	1.66e-4	4685.756
Disable meta learning and reduce TCN depth	2.62e-5	3.21e-4	0.709	0.912	0.755	0.826	1.64e-4	**1283.069**

8.3 Lightweight Cleaner evaluation

Table 9 reports the effectiveness-efficiency tradeoff of lightweight Cleaner. Since the dynamic scheduling of Cleaner relies on meta-learning adaptation and TCN-based feature extraction, they introduce additional computational overhead. To explore efficiency optimization, we consider two simplified versions: (1) disabling meta learning, (2) further reducing TCN depth. The lightweight designs greatly reduce computational cost while maintaining competitive cleaning performance. In particular, reducing TCN depth achieves about a 3.65× speedup over the version without meta learning, with comparable or slightly better performance on some metrics, making lightweight Cleaner suitable for resource-constrained scenarios.

8.4 Sensitivity on model-level parameters and design

8.4.1 Window sizes l. We investigate the impact of l on Cleaner in Table 10. When l is small, the performance is relatively limited due to insufficient temporal context. As l increases, the performance consistently improves. However, overly large windows may introduce redundant data information and weaken the effectiveness of cleaning policy. Since the best overall performance is achieved at a window size of 200 across datasets, we adopt it as the default setting.

8.4.2 Quality-termination threshold ϵ and maximum iteration budget C. We study the impact of ϵ and C on Cleaner under OilTemp and Power in Figure 6. A large ϵ triggers early termination and insufficient cleaning, while a small ϵ improves quality with higher cost. Similarly, a small C limits cleaning depth, whereas a large C

Table 10: The influence of window sizes on the effectiveness of Cleaner

Methods	Windows	IDF							Power_real							OilTemp_real						
		MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓
Cleaner	50	3.98e-5	4.81e-4	0.594	0.819	0.682	0.744	2.11e-4	1.81e-2	1.10e-2	0.415	0.452	0.403	0.426	6.35e-3	2.73e-4	2.52e-3	0.795	**0.983**	0.586	0.734	2.02e-3
	100	3.43e-5	3.48e-4	0.678	0.842	0.725	0.779	1.89e-4	2.55e-3	5.46e-3	0.725	0.635	0.615	0.625	3.06e-3	2.22e-4	2.75e-3	0.790	0.980	0.604	0.747	1.67e-3
	160	4.25e-5	4.13e-4	0.629	0.844	0.676	0.751	2.07e-4	2.51e-3	6.39e-3	0.712	0.596	0.589	0.593	2.37e-3	9.60e-4	6.07e-3	0.574	0.644	0.503	0.564	2.75e-3
	200	**1.51e-5**	**1.36e-4**	**0.720**	**0.949**	**0.801**	**0.868**	**8.74e-5**	**1.01e-3**	**1.49e-3**	**0.931**	**0.903**	**0.897**	**0.899**	**1.32e-3**	**1.98e-4**	**1.56e-3**	**0.895**	0.976	**0.751**	**0.848**	1.45e-3
	250	4.51e-5	5.35e-4	0.565	0.807	0.687	0.742	2.15e-4	3.01e-3	6.65e-3	0.687	0.653	0.611	0.631	2.97e-3	2.33e-4	3.13e-3	0.772	0.979	0.629	0.766	**1.44e-3**

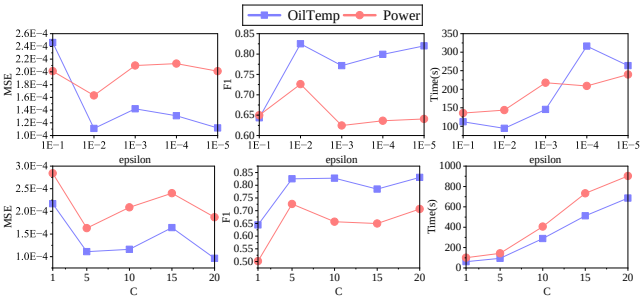


Fig. 6: Analysis of threshold ϵ and max iteration C

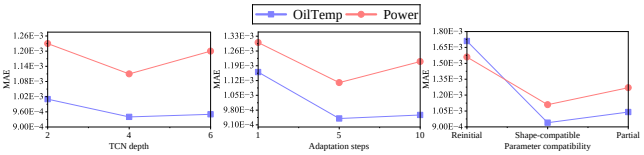


Fig. 7: Influence of various parameters

brings diminishing gains with increased runtime. We set $\epsilon = 10^{-2}$ and $C = 5$ to balance effectiveness and efficiency.

8.4.3 Other parameters. We evaluate the impact of TCN depth, adaptation steps, and parameter compatibility in Figure 7. The MAE results show that the default settings, corresponding to the middle x-axis values, generally achieve the best performance, while small variations across different settings indicate that Cleaner is robust to parameter changes.

8.4.4 Reference quality evaluation. We assess the impact of reference quality on Cleaner under five settings in Table 11. By default, Cleaner relies on reference data for scheduling. *High-quality historical reference* achieves performance close to the *Oracle reference*, validating Cleaner's reference estimation mechanism. As reference quality deteriorates, performance gradually degrades, especially under *Low-quality historical reference* and *Cross-dataset reference*, due to less reliable reward estimation and termination decisions. This observation aligns with the properties in §5, where the error bound of cleaned results is dependent on the reference error and becomes less restrictive when reference quality deteriorates.

When reference data are unavailable, we introduce the *No-reference* setting, which uses the TCN reconstruction loss as an alternative learning signal. Specifically, a TCN model trained on clean data computes the reconstruction loss before and after each action, and the change in reconstruction loss is used to determine the reward. For cleaning and repair actions, rewards reflect the reduction in reconstruction loss; for detection actions, detected errors are temporarily replaced with column-wise means, and rewards are computed based on the resulting change in reconstruction loss. The results show that the *No-reference* setting achieves relatively reliable cleaning performance, although its effectiveness varies across datasets.

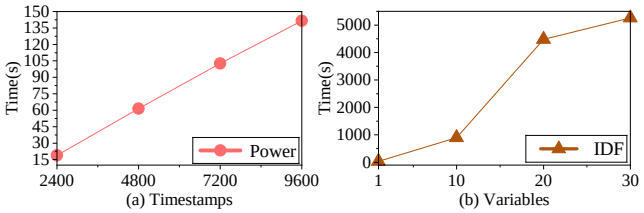


Fig. 8: Influence of timestamp T and variable M

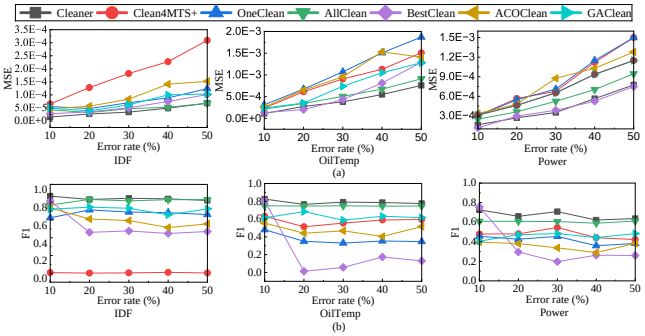


Fig. 9: The cleaning effectiveness at different error rates

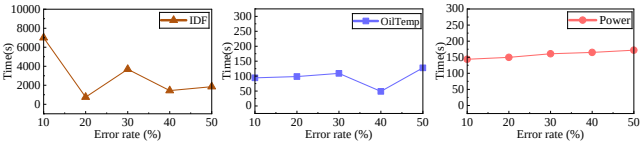


Fig. 10: Influence of error rates

Overall, the default reference-based mechanism performs better when reference data are available, while the *No-reference* setting provides a viable alternative for autonomous cleaning when such data are unavailable.

8.5 Study on data characteristics

8.5.1 Time series length T and variable number M. We analyze the impact of T and M on time cost in Figure 8. As T increases from 2400 to 9600, the time cost on Power grows approximately linearly, consistent with the efficiency analysis in §7. As M increases from 1 to 30 on IDF, the growth of time cost gradually slows, demonstrating the scalability of Cleaner.

8.5.2 Error rates. We investigate the influence of error rates in Figure 9. Cleaner consistently outperforms or matches baselines across datasets and metrics, achieving average improvements of 43% and 35% in MSE and F1, respectively. We assess the time cost of Cleaner under varying error rates in Figure 10. Its time cost remains stable or decreases with increasing error rates, indicating faster convergence under denser errors.

Table 11: The effectiveness of reference quality in CLEANER

Methods	IDF							OilTemp							Power						
	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓
Oracle reference (ground truth)	1.81e-5	2.39e-4	**0.772**	**0.977**	0.795	**0.877**	1.48e-4	**9.77e-5**	**1.40e-3**	**0.717**	0.967	**0.727**	**0.830**	**9.14e-4**	**1.05e-4**	**1.31e-3**	**0.648**	**0.950**	**0.693**	**0.802**	**9.67e-4**
High-quality historical reference (Same-source clean history)	**1.51e-5**	**1.36e-4**	0.720	0.949	**0.801**	0.868	**8.74e-5**	1.11e-4	1.59e-3	0.701	**0.971**	0.717	0.825	9.40e-4	1.63e-4	1.59e-3	0.645	0.854	0.632	0.726	1.11e-3
Low-quality historical reference (Same-source distribution shift)	4.99e-5	5.73e-4	0.566	0.732	0.735	0.733	1.84e-4	2.31e-4	2.92e-3	0.520	0.834	0.660	0.737	1.16e-3	2.24e-4	2.20e-3	0.493	0.661	0.608	0.633	1.11e-3
Cross-dataset reference (Cross-dataset clean segments)	3.63e-5	4.63e-4	0.625	0.828	0.735	0.779	1.71e-4	1.26e-4	1.71e-3	0.670	0.931	0.689	0.792	9.79e-4	2.14e-4	2.06e-3	0.515	0.670	0.587	0.626	1.09e-3
No reference (TCN reconstruction loss)	4.00e-5	5.07e-4	0.538	0.862	0.621	0.722	2.50e-4	1.70e-4	2.50e-3	0.571	0.941	0.689	0.796	1.07e-3	1.45e-4	1.64e-3	0.577	0.880	0.614	0.724	1.07e-3

Table 12: Influence of evolving errors

Methods	OilTemp		Power	
	MAE ↓	F1 ↑	MAE ↓	F1 ↑
OneClean	3.49e-3	0.509	5.06e-3	0.421
AllClean	1.93e-3	0.681	3.20e-3	0.514
BestClean	2.18e-3	0.835	2.99e-3	**0.806**
GAClean	2.86e-3	0.676	3.42e-3	0.656
PSOClean	2.60e-3	0.720	3.34e-3	0.740
ACOClean	3.05e-3	0.685	4.80e-3	0.537
DEClean	1.92e-3	0.813	4.41e-3	0.597
DemandClean	1.39e-1	0.214	3.65e-2	0.246
CLEANER-MAB	1.88e-3	0.844	4.04e-3	0.538
CLEANER-DQN	1.98e-3	0.812	3.02e-3	0.726
CLEANER	**1.55e-3**	**0.858**	**2.59e-3**	0.764

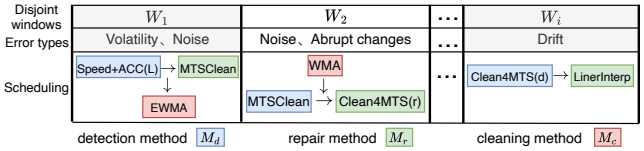


Fig. 11: Scheduling visualization on evolving errors

Table 13: The generalization to new datasets and error types

Settings	OilTemp			Power		
	MSE ↓	F1 ↑	Adapt/Retrain Time (s) ↓	MSE ↓	F1 ↑	Adapt/Retrain Time (s) ↓
Zero-Shot Transfer	1.92e-4	0.675	–	2.65e-4	0.573	–
Target-Only Training	**1.08e-4**	0.817	364.780	**1.60e-4**	**0.731**	550.950
Meta-based Adaptation (ours)	1.11e-4	**0.825**	**40.838**	1.63e-4	0.726	**51.392**

8.5.3 Evolving errors. We evaluate the impact of evolving errors on cleaning performance by dividing each time series into 10 consecutive windows, injecting three error types into the first five windows and two different types into the remaining five. Table 12 compares CLEANER with baselines on OilTemp and Power, while Figure 11 visualizes its cleaning schedules across windows. The results show that CLEANER effectively handles evolving errors, outperforming most baselines and achieving comparable or better performance than BestClean in some cases. The visualization illustrates its adaptive scheduling: it prefers smoothing methods from the coupled paradigm for noise-dominated windows and constraint-based methods from the decoupled paradigm for abrupt changes or drifts.

8.6 CLEANER adaptation

8.6.1 Adaptation to new datasets and error types. We validate the generalization of CLEANER to new datasets under three settings: (1) *Meta-based Adaptation (ours)*, which meta-trains on $N-1$ source datasets and adapts to the target dataset; (2) *Zero-Shot Transfer*, which directly transfers the source-trained model to the target dataset without adaptation; (3) *Target-Only Training*, which trains from scratch on the target dataset and serve as an upper bound. Table 13 shows *Meta-based Adaptation* consistently beats *Zero-Shot Transfer* and approaches *Target-Only Training* at far lower cost.

8.6.2 Adaptation to new methods. We evaluate the generalization of CLEANER to new methods under three settings: (1) *Training Methods Only*, using existing methods; (2) *Directly Apply New Methods*, adding M_{new} without adaptation; and (3) *Adapt New Methods (ours)*, adapting to M_{new}, as shown in Table 14. Compared with *Training Methods Only*, *Directly Apply New Methods* improves performance but provides limited gains without adaptation. In contrast, *Adapt New Methods (ours)* achieves further improvements, demonstrating that adaptation enables CLEANER to effectively exploit new methods beyond the predefined set.

Table 14: The generalization to new methods

Settings	OilTemp			Power		
	New Methods	MSE ↓	F1 ↑	New Methods	MSE ↓	F1 ↑
Training Methods Only	–	1.87e-4	0.736	–	2.41e-4	0.560
Directly Apply New Methods	+ M_{new}	1.35e-4	0.803	+ M_{new}	2.05e-4	0.638
Adapt New Methods (ours)	+ M_{new}	**1.11e-4**	**0.825**	+ M_{new}	**1.63e-4**	**0.726**

8.7 Ablation study

We evaluate the contributions of the *Cleaning Path*, *Discriminator*, and *Adaptation Mechanism* in CLEANER through ablation studies.

8.7.1 Cleaning Path. We investigate the effectiveness of cleaning agents and TVACN in the Cleaning Path, as shown in Table 15. Removing either $\mathcal{A}_c$ or $\mathcal{A}_{d+r}$ degrades overall cleaning performance, demonstrating their complementary roles in handling heterogeneous errors. Replacing TVACN with two alternative RL strategies, i.e., CLEANER-MAB and CLEANER-DQN, results in inferior performance, indicating the effectiveness of the proposed cleaning policy. Collectively, these results verify the necessity of jointly modeling diverse cleaning paradigms and dynamic method scheduling.

8.7.2 Discriminator. We evaluate the necessity of evidence components in the Discriminator. As shown in Table 15, removing any single evidence component leads to performance degradation and less reliable termination decisions, demonstrating that jointly considering multiple complementary evidence signals is crucial for reducing missed error and over-cleaning.

8.7.3 Adaptation Mechanism. We evaluate the effectiveness of Knowledge Integration (KI) and Model Adaptation (MA) in the Adaptation Mechanism. As in Table 15, removing KI or MA significantly impairs cross-dataset knowledge transfer and adaptation to unseen datasets, confirming the necessity of both components.

8.8 Downstream performance

8.8.1 Task utility. We evaluate the utility of cleaned data on three downstream tasks, including regression (Fig. 12(a–c)), clustering (Fig. 12(d–f)), and time-series forecasting (Fig. 12(g–i)). Regression is evaluated using MSE with an autoencoder, clustering uses K-means with SS (see §8.1), and forecasting is evaluated by predicting future values from historical observations. "Clean" and "Dirty" denote original clean and corrupted data, respectively. CLEANER consistently outperforms baselines and approaches the performance of "Clean" data, demonstrating its effectiveness in restoring data utility for downstream analysis.

8.8.2 Refeed evaluation. We evaluate the cleaning quality by refeeding cleaned data into the detector on real datasets and comparing

Table 15: Ablation Study of Cleaner

Component	Methods	IDF							Power_real							OilTemp_real						
		MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓	MSE ↓	MNAD ↓	RRA ↑	P ↑	R ↑	F1 ↑	MAE ↓
Cleaning Path	*w/o* $\mathcal{A}_c$	3.60e-5	2.87e-4	0.659	0.879	0.690	0.773	2.10e-4	3.60e-3	2.11e-3	0.852	0.721	0.799	0.758	1.95e-3	2.25e-4	2.44e-3	0.797	0.967	0.665	0.786	1.61e-3
	w/o $\mathcal{A}_{d+r}$	1.63e-5	1.41e-4	0.701	0.932	0.661	0.774	1.07e-4	2.35e-3	5.38e-3	0.752	0.832	0.679	0.748	2.20e-3	3.16e-4	2.61e-3	0.791	0.884	0.628	0.733	1.99e-3
	Cleaner-MAB	4.80e-5	4.67e-4	0.610	0.783	0.709	0.744	1.88e-4	1.11e-3	5.75e-3	0.759	0.654	0.695	0.674	**1.21e-3**	2.29e-4	2.78e-3	0.788	0.965	0.611	0.748	1.57e-3
	Cleaner-DQN	3.19e-5	3.14e-4	0.691	**0.976**	0.692	0.809	2.02e-4	1.22e-3	5.09e-3	0.773	0.784	0.693	0.735	1.57e-3	2.42e-4	2.77e-3	0.789	0.951	0.609	0.742	1.58e-3
	Cleaner	**1.51e-5**	**1.36e-4**	**0.720**	0.949	**0.801**	**0.868**	**8.74e-5**	**1.01e-3**	**1.49e-3**	**0.931**	**0.903**	**0.897**	**0.899**	1.32e-3	**1.98e-4**	**1.56e-3**	**0.895**	**0.976**	**0.751**	**0.848**	**1.45e-3**
Discriminator	*w/o* Dones	3.21e-5	3.24e-4	0.694	0.920	0.693	0.791	1.91e-4	2.20e-3	5.18e-3	0.754	0.807	0.655	0.723	2.34e-3	5.12e-4	3.83e-3	0.706	0.883	0.572	0.694	2.18e-3
	w/o Step	2.69e-5	3.00e-4	0.717	0.938	0.751	0.834	1.78e-4	1.09e-3	4.21e-3	0.802	0.799	0.623	0.700	1.77e-3	2.94e-4	3.00e-3	0.768	0.947	0.607	0.740	1.80e-3
	Cleaner	**1.51e-5**	**1.36e-4**	**0.720**	**0.949**	**0.801**	**0.868**	**8.74e-5**	**1.01e-3**	**1.49e-3**	**0.931**	**0.903**	**0.897**	**0.899**	**1.32e-3**	**1.98e-4**	**1.56e-3**	**0.895**	**0.976**	**0.751**	**0.848**	**1.45e-3**
Adaptation Mechanism	*w/o* KI	3.47e-5	2.94e-4	0.692	**0.950**	0.634	0.760	2.26e-4	2.28e-3	3.74e-3	0.806	0.875	0.594	0.708	2.54e-3	9.38e-4	4.95e-3	0.573	0.791	0.412	0.542	3.42e-3
	w/o MA	2.90e-5	3.28e-4	0.694	0.920	0.721	0.809	1.87e-4	2.25e-3	4.76e-3	0.768	0.745	0.635	0.686	2.42e-3	4.35e-4	3.64e-3	0.717	0.926	0.565	0.702	2.16e-3
	Cleaner	**1.51e-5**	**1.36e-4**	**0.720**	0.949	**0.801**	**0.868**	**8.74e-5**	**1.01e-3**	**1.49e-3**	**0.931**	**0.903**	**0.897**	**0.899**	**1.32e-3**	**1.98e-4**	**1.56e-3**	**0.895**	**0.976**	**0.751**	**0.848**	**1.45e-3**

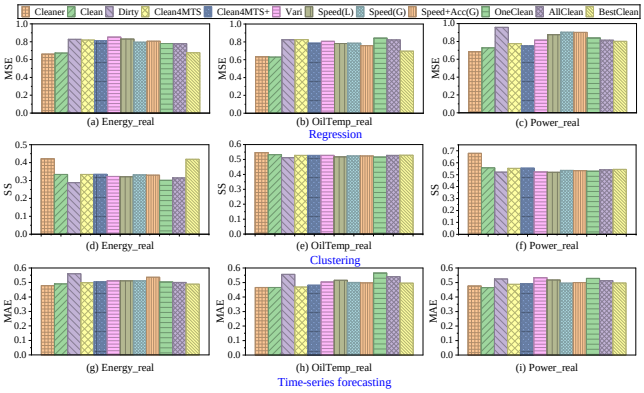


Fig. 12: Downstream task evaluation

Table 16: The quality gains from refeeding cleaned data

Methods	Forms	Energy_real	OilTemp_real	Power_real
		F1	F1	F1
Clean4MTS	Original input	0.917	**0.908**	0.873
	Refeeding input (Relative improvement)	0.117 (0.872)	0.009 (0.990)	**0** (**1**)
Cleaner	Original input	**0.922**	0.848	**0.899**
	Refeeding input (Relative improvement)	**0.016** (**0.983**)	**0.004** (**0.995**)	0.008 (0.991)

Cleaner with the strongest baseline (Clean4MTS [7]), in Table 16. "Relative improvement" measures the reduction in detected errors after cleaning. Cleaner consistently reduces re-detected errors across various datasets, showing that its adaptive pipeline construction and scheduling effectively improve cleaning quality. It also outperforms the strongest baseline in most cases, demonstrating the advantage of autonomous cleaning over fixed cleaning strategies.

9 Related work

Single-error cleaning methods. Existing methods target specific error types using smoothing [2, 17], constraint-based (e.g., column-wise [39, 42, 48] and row-wise [16]), or statistical approaches [45, 50]. However, they rely on single-error assumptions and are limited when heterogeneous errors co-occur.

Multi-error cleaning methods. Constraint-based methods [7, 10, 52] combine column- and row-wise dependencies to handle diverse errors. Statistical methods (e.g., Akane) [22] formulate cleaning as cost-constrained perplexity minimization. System-level methods (L2C [24], Clean4TSDB [12], Cleanits [14]) and learning-based methods (e.g., MEDetect [11]) integrate multiple modules, and tabular workflow optimizers (UniClean [8, 9], CBAClean [13], DemandClean [36], and RLclean [35]) compose cleaning operations. However, most assume fixed error types and predefined integration strategies, limiting adaptability to evolving errors.

Temporal-adaptive cleaning methods. SAX [33] repairs missing values in real time via symbolic aggregate approximation. Speed [42] and CDDC [6] correct single-point or consecutive errors online with (dynamic) speed constraints. These methods adapt over time but remain bound to a single error type and cannot schedule or compose diverse cleaning paradigms as heterogeneous errors evolve.

AutoML-based cleaning pipelines. AutoML-based methods automate the selection, ordering, or composition of data cleaning and preprocessing operations. AlphaClean [27] and BoostClean [26] optimize cleaning decisions based on downstream model utility, while DiffPrep [28] and AutoClean [31] extend automation to broader data preparation and workflow optimization. Despite their automation capabilities, these methods primarily focus on general-purpose data preparation and do not explicitly address the joint challenges of evolving temporal patterns and heterogeneous errors. In contrast, Cleaner integrates window-level scheduling with time-series-specific operators, coordinates coupled and decoupled cleaning, and incorporates adaptive termination to mitigate premature stopping and over-cleaning, enabling fine-grained, temporally adaptive cleaning.

10 Conclusion

This paper initiates the ASC problem and proposes Cleaner, an autonomous cleaning framework that adaptively schedules diverse TSC methods for heterogeneous and evolving errors. Cleaner constructs a Cleaning Path that integrates coupled and decoupled cleaning paradigms, and leverages RL with a Discriminator and an Adaptation Mechanism for adaptive selection, termination, and adaptation. Extensive experiments validate its superiority, and future work will explore Cleaner to large-scale time series scenarios.

References

[1] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. 2000. LOF: Identifying Density-Based Local Outliers. In Proceedings of the ACM on Management of Data. ACM, New York, NY, USA, 93–104. doi:10.1145/335191.335388

[2] David R Brillinger. 2001. Time series: data analysis and theory. SIAM, Philadelphia, PA, USA. doi:10.1137/1.9780898719246

[3] Varun Chandola, Arindam Banerjee, and Vipin Kumar. 2009. Anomaly detection: A survey. ACM computing surveys (CSUR) 41, 3 (2009), 1–58. doi:10.1145/1541880.1541882

[4] Zijie Chen, Aoqian Zhang, and Shaoxu Song. 2025. Cleaning Time Series under Seasonal and Trend Constraints. Proceedings of the ACM on Management of Data 3, 6 (2025), 1–26. doi:10.1145/3769773

[5] Tamraparni Dasu and Ji Meng Loh. 2012. Statistical Distortion: Consequences of Data Cleaning. Proceedings of the VLDB Endowment 5, 11 (2012), 1674–1683. doi:10.14778/2350229.2350279

[6] Guohui Ding, Chenyang Li, Ru Wei, Shasha Sun, Zhaoyu Liu, and Chunlong Fan. 2020. Time Series Data Cleaning Based on Dynamic Speed Constraints. In International Conference on Web Information Systems Engineering. Springer, Cham, Switzerland, 475–487. doi:10.1007/978-3-030-62008-0_33

[7] Xiaoou Ding, Genglong Li, Hongzhi Wang, Chen Wang, and Yichen Song. 2024. Time series data cleaning under expressive constraints on both rows and columns. In International Conference on Data Engineering. IEEE, Piscataway, NJ, USA, 5689–5698. doi:10.1109/ICDE60146.2024.00283

[8] Xiaoou Ding, Zekai Qian, Hongzhi Wang, Siying Chen, Yafeng Tang, Hongbin Su, Huan Hu, and Chen Wang. 2025. UniClean: A Scalable Data Cleaning Solution for Mixed Errors based on Unified Cleaners and Optimized Cleaning Workflow. Proceedings of the VLDB Endowment 18, 11 (2025), 4117–4130. doi:10.14778/3749646.3749681

[9] Xiaoou Ding, Zekai Qian, Hongzhi Wang, Zhe Sun, Siying Chen, Hongbin Su, and Huan Hu. 2025. UniClean: A Multi-Signal Fusion Pipeline for Optimizing Data Cleaning Workflow. In International Conference on Data Engineering. IEEE, Piscataway, NJ, USA, 4600–4603. doi:10.1109/ICDE65448.2025.00362

[10] Xiaoou Ding, Yichen Song, Hongzhi Wang, Chen Wang, and Donghua Yang. 2024. MTSClean: Efficient Constraint-based Cleaning for Multi-Dimensional Time Series Data. Proceedings of the VLDB Endowment 17, 13 (2024), 4840–4852. doi:10.14778/3704965.3704987

[11] Xiaoou Ding, Yichen Song, Hongzhi Wang, Donghua Yang, and Yida Liu. 2023. Cleanits-MEDetect: Multiple Errors Detection for Time Series Data in Cleanits. In Database Systems for Advanced Applications. Springer, Cham, Switzerland, 674–678. doi:10.1007/978-3-031-30678-5_54

[12] Xiaoou Ding, Yichen Song, Hongzhi Wang, Donghua Yang, Chen Wang, and Jianmin Wang. 2024. Clean4TSDB: A Data Cleaning Tool for Time Series Databases. Proceedings of the VLDB Endowment 17, 12 (2024), 4377–4380. doi:10.14778/3685800.3685879

[13] Xiaoou Ding, Hongbin Su, Zekai Qian, Wenxuan Cui, Siying Chen, Zheng Liang, Chen Wang, and Hongzhi Wang. 2025. CBAClean: A Comprehensive System for Recommending Data Cleaning Solutions Through Cost-Benefit Analysis in Data Quality Management. In International Conference on Data Engineering. IEEE, Piscataway, NJ, USA, 4636–4639. doi:10.1109/ICDE65448.2025.00371

[14] Xiaoou Ding, Hongzhi Wang, Jiaxuan Su, Zijue Li, Jianzhong Li, and Hong Gao. 2019. Cleanits: A Data Cleaning System for Industrial Time Series. Proceedings of the VLDB Endowment 12, 12 (2019), 1786–1789. doi:10.14778/3352063.3352066

[15] Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. 1996. Ant system: optimization by a colony of cooperating agents. IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics) 26, 1 (1996), 29–41. doi:10.1109/3477.484436

[16] Anna Fariha, Ashish Tiwari, Arjun Radhakrishna, Sumit Gulwani, and Alexandra Meliou. 2021. Conformance constraint discovery: Measuring trust in data-driven systems. In Proceedings of the ACM on Management of Data. ACM, New York, NY, USA, 499–512. doi:10.1145/3448016.3452795

[17] Roland Fried and Ann Cathrice George. 2011. Exponential and Holt-Winters Smoothing. doi:10.1007/978-3-642-04898-2_244

[18] Everette S. Gardner. 2006. Exponential smoothing: The state of the art—Part II. International Journal of Forecasting 22, 4 (2006), 637–666. doi:10.1016/j.ijforecast.2006.03.005

[19] Lukasz Golab, Howard Karloff, Flip Korn, Avishek Saha, and Divesh Srivastava. 2009. Sequential dependencies. Proceedings of the VLDB Endowment 2, 1 (2009), 574–585. doi:10.14778/1687627.1687693

[20] Gary K Grunwald and Rob J Hyndman. 1998. Smoothing non-Gaussian time series with autoregressive structure. Computational statistics & data analysis 28, 2 (1998), 171–191. doi:10.1016/S0167-9473(98)00034-6

[21] Min Han and Qipeng Wang. 2025. Adaptive graph convolution neural differential equation for spatio-temporal time series prediction. IEEE Transactions on Knowledge and Data Engineering 37, 6 (2025), 3193–3204. doi:10.1109/TKDE.2024.3383895

[22] Xiaoyu Han, Haoran Xiong, Zhenying He, Peng Wang, Chen Wang, and X. Sean Wang. 2024. Akane: Perplexity-Guided Time Series Data Cleaning. Proceedings of the ACM on Management of Data 2, 3 (2024), 121. doi:10.1145/3654993

[23] Joseph M. Hellerstein. 2008. Quantitative Data Cleaning for Large Databases. https://dsf.berkeley.edu/jmh/papers/cleaning-unece.pdf

[24] Mayuresh Hooli and Rabi Mahapatra. 2025. L2C: Learn to Clean Time Series Data. In International Conference on Internet of Things, Big Data and Security. SciTePress, Setúbal, Portugal, 361–369. doi:10.5220/0013421300003944

[25] Eamonn J. Keogh, Selina Chu, David M. Hart, and Michael J. Pazzani. 2001. An Online Algorithm for Segmenting Time Series. In International Conference on Data Mining. IEEE, Piscataway, NJ, USA, 289–296. doi:10.1109/ICDM.2001.989531

[26] Sanjay Krishnan, Michael J Franklin, Ken Goldberg, and Eugene Wu. 2017. Boostclean: Automated error detection and repair for machine learning. arXiv preprint arXiv:1711.01299 (2017). doi:10.48550/arXiv.1711.01299

[27] Sanjay Krishnan and Eugene Wu. 2019. Alphaclean: Automatic generation of data cleaning pipelines. arXiv preprint arXiv:1904.11827 (2019). doi:10.48550/arXiv.1904.11827

[28] Peng Li, Zhiyi Chen, Xu Chu, and Kexin Rong. 2023. Diffprep: Differentiable data preprocessing pipeline search for learning over tabular data. Proceedings of the ACM on Management of Data 1, 2 (2023), 1–26. doi:10.1145/3589328

[29] Zheng Liang, Hongzhi Wang, Xiaoou Ding, and Tianyu Mu. 2021. Industrial time series determinative anomaly detection based on constraint hypergraph. Knowledge-Based Systems 233 (2021), 107548. doi:10.1016/j.knosys.2021.107548

[30] Zudi Lu and Y.V. Hui. 2003. L1 linear interpolator for missing values in time series. Annals of the Institute of Statistical Mathematics 55, 1 (2003), 197–216. doi:10.1007/BF02530494

[31] Felix Neutatz, Binger Chen, Yazan Alkhatib, Jingwen Ye, and Ziawasch Abedjan. 2022. Data Cleaning and AutoML: Would an optimizer choose to clean? Datenbank-Spektrum 22, 2 (2022), 121–130. doi:10.1007/S13222-022-00413-2

[32] Alex Nichol and John Schulman. 2018. Reptile: a Scalable Metalearning Algorithm. arXiv preprint arXiv:1803.02999 (2018). doi:10.48550/arXiv.1803.02999

[33] Xuefeng Ning. 2022. Data Cleaning Technology of Substation Internet of Things Based on Time Series Association. In International Conference on Communication Technology. IEEE, Piscataway, NJ, USA, 899–904. doi:10.1109/ICCT56141.2022.10073228

[34] Andrey T. Palma, Vania Bogorny, Bart Kuijpers, and Luis Otavio Alvares. 2008. A Clustering-Based Approach for Discovering Interesting Places in Trajectory Data. In ACM Symposium on Applied Computing. ACM, New York, NY, USA, 863–868. doi:10.1145/1363686.1363886

[35] Jinfeng Peng, Derong Shen, Tiezheng Nie, and Yue Kou. 2024. RLclean: An unsupervised integrated data cleaning framework based on deep reinforcement learning. Information Sciences 682 (2024), 121281. doi:10.1016/J.INS.2024.121281

[36] Zekai Qian, Xiaoou Ding, Chen Wang, and Hongzhi Wang. 2025. DemandClean: A Multi-Objective Learning Framework for Balancing Model Tolerance to Data Authenticity and Diversity. Proceedings of the VLDB Endowment 18, 12 (2025), 5339–5342. doi:10.14778/3750601.3750666

[37] Chowdhury K R. 2020. KNNImputer: A robust way to impute missing values (using Scikit-Learn). https://scikit-learn.org/stable/modules/generated/sklearn.impute.KNNImputer.html

[38] Aleksandrs Slivkins. 2019. Introduction to Multi-Armed Bandits. Foundations and Trends in Machine Learning 12, 1-2 (2019), 1–286. doi:10.1561/2200000068

[39] Shaoxu Song, Fei Gao, Aoqian Zhang, Jianmin Wang, and Philip S Yu. 2021. Stream data cleaning under speed and acceleration constraints. ACM Transactions on Database Systems (TODS) 46, 3 (2021), 1–44. doi:10.1145/3465740

[40] Shaoxu Song, Yu Sun, Aoqian Zhang, Lei Chen, and Jianmin Wang. 2020. Enriching Data Imputation under Similarity Rule Constraints. IEEE Transactions on Knowledge and Data Engineering 32, 2 (2020), 275–287. doi:10.1109/TKDE.2018.2883103

[41] Shaoxu Song and Aoqian Zhang. 2020. IoT Data Quality. In Proceedings of the ACM International Conference on Information and Knowledge Management. ACM, New York, NY, USA, 3517–3518. doi:10.1145/3340531.3412173

[42] Shaoxu Song, Aoqian Zhang, Jianmin Wang, and Philip S Yu. 2015. SCREEN: Stream data cleaning under speed constraints. In Proceedings of the ACM on Management of Data. ACM, New York, NY, USA, 827–841. doi:10.1145/2723372.2723730

[43] Rainer Storn and Kenneth Price. 1997. Differential evolution–a simple and efficient heuristic for global optimization over continuous spaces. Journal of Global Optimization 11, 4 (1997), 341–359. doi:10.1023/A:1008202821328

[44] Kok Cheng Tan, Daniel Zantedeschi, Amruth Kumar, and Alessio Gaspar. 2022. Genetic algorithm cleaning in sequential data mining: analyzing solutions to parsons' puzzles. In Proceedings of the Genetic and Evolutionary Computation Conference Companion. ACM, New York, NY, USA, 2330–2333. doi:10.1145/3520304.3534025

[45] Haoyu Wang, Aoqian Zhang, Shaoxu Song, and Jianmin Wang. 2024. Streaming data cleaning based on speed change. The VLDB Journal 33, 1 (2024), 1–24. doi:10.1007/S00778-023-00796-Y

[46] Xi Wang and Chen Wang. 2019. Time series data cleaning: A survey. IEEE Access 8 (2019), 1866–1881. doi:10.1109/ACCESS.2019.2962152

[47] Jianzhuo Yan, Xinyue Chen, and Yongchuan Yu. 2018. A Data Cleaning Framework for Water Quality based on NLDIW-PSO based optimal SVR. In International Conference on Web Intelligence. IEEE, Piscataway, NJ, USA, 336–341. doi:10.1109/WI.2018.00-71

[48] Wei Yin, Tianbai Yue, Hongzhi Wang, Yanhao Huang, and Yaping Li. 2018. Time series cleaning under variance constraints. In Database Systems for Advanced Applications. Springer, Cham, Switzerland, 108–113. doi:10.1007/978-3-319-91455-8_10

[49] Aoqian Zhang, Shuqing Deng, Dongping Cui, Ye Yuan, and Guoren Wang. 2023. An Experimental Evaluation of Anomaly Detection in Time Series. Proceedings of the VLDB Endowment 17, 3 (2023), 483–496. doi:10.14778/3632093.3632110

[50] Aoqian Zhang, Shaoxu Song, and Jianmin Wang. 2016. Sequential data cleaning: A statistical approach. In Proceedings of the ACM on Management of Data. ACM, New York, NY, USA, 909–924. doi:10.1145/2882903.2915233

[51] Aoqian Zhang, Shaoxu Song, Jianmin Wang, and Philip S Yu. 2017. Time series data cleaning: From anomaly detection to anomaly repairing. Proceedings of the VLDB Endowment 10, 10 (2017), 1046–1057. doi:10.14778/3115404.3115410

[52] Aoqian Zhang, Zexue Wu, Yifeng Gong, Ye Yuan, and Guoren Wang. 2024. Multivariate Time Series Cleaning under Speed Constraints. Proceedings of the ACM on Management of Data 2, 6 (2024), 1–26. doi:10.1145/3698821

[53] OUYANG Zhenzheng, Zhao Zipeng, Gao Yuhai, and Wang Tao. 2011. Study on the classification of data streams with concept drift. International Conference on Fuzzy Systems and Knowledge Discovery 3 (2011), 1673–1677. doi:10.1109/

FSKD.2011.6019889

[54] Jingyu Zhu, Weiwei Deng, Yu Sun, Shaoxu Song, Haiwei Zhang, and Xiaojie Yuan. 2025. The Best of Both Worlds: On Repairing Timestamps and Attribute Values for Multivariate Time Series. Proceedings of the ACM on Management of Data 3, 3 (2025), 1–27. doi:10.1145/3725367

[55] Yongzhen Zhuang, Lei Chen, X Sean Wang, and Jie Lian. 2007. A weighted moving average-based approach for cleaning sensor data. In International Conference on Distributed Computing Systems. IEEE, Piscataway, NJ, USA, 38. doi:10.1109/ICDCS.2007.83